```
)

print(matriz_cor)
```

Cómo leer este código

El argumento:

Terminal

```
use = "pairwise.complete.obs"
```

indica que cada correlación se calcula utilizando las observaciones disponibles para ese par de variables.

Buena práctica

Cuando se utiliza una estrategia para trabajar con valores faltantes, esa decisión debe quedar explícita. Diferentes pares de variables pueden terminar utilizando cantidades distintas de observaciones.

14.8. Regresión lineal simple

Supongamos que queremos utilizar la humedad relativa media para describir parte de la variación de la temperatura media.

Un modelo lineal simple puede escribirse como:

$$y = \beta_0 + \beta_1 x + \varepsilon.$$

En nuestro ejemplo:

- y corresponde a la temperatura media;
- x corresponde a la humedad relativa media;
- β_0 es el intercepto;
- β_1 representa el cambio esperado en y asociado a una unidad de cambio en x;
- ε representa la parte no explicada por el modelo.

En R utilizamos `lm`:

Ajustar una regresión lineal

```
modelo_simple <- lm(
    temp_media ~ hr_media,
    data = temp_humedad
)

print(modelo_simple)
```

Cómo leer este código

La fórmula:

Terminal

```
temp_media ~ hr_media
```

puede leerse como:

modelar `temp_media` en función de `hr_media`.

14.9. Consultar los coeficientes

Podemos consultar directamente los coeficientes:

Coeficientes del modelo

```
coef(modelo_simple)
```

También podemos guardarlos:

Guardar coeficientes

```
beta_0 <- coef(modelo_simple)[1]
beta_1 <- coef(modelo_simple)[2]

print(beta_0)
print(beta_1)
```

El modelo ajustado tiene entonces la forma:

$$\widehat{y} = \widehat{\beta}_0 + \widehat{\beta}_1 x.$$

Idea clave

La pendiente debe interpretarse utilizando las unidades de las variables.

Por ejemplo, si `hr_media` está expresada en puntos porcentuales, la pendiente representa el cambio estimado en temperatura asociado a un aumento de un punto porcentual en humedad, manteniendo el contexto del modelo utilizado.

14.10. Obtener un resumen del modelo

La función `summary` muestra información más completa:

Resumen de la regresión

```
summary(modelo_simple)
```

Entre los elementos que aparecen podemos encontrar:

- estimaciones de los coeficientes;
- errores estándar;
- estadísticos asociados a los coeficientes;
- residuos;
- coeficiente de determinación R^2;
- R^2 ajustado.

En esta unidad curricular nos concentraremos principalmente en la interpretación práctica del modelo, sus predicciones y sus errores.

Idea clave

Un modelo no debe evaluarse solamente por un único número obtenido de `summary`. También debemos revisar su comportamiento sobre los datos.

14.11. El coeficiente de determinación

El valor R^2 indica qué proporción de la variación observada de la variable respuesta es explicada por el modelo, dentro del conjunto utilizado para ajustarlo.

Podemos obtenerlo mediante:

Obtener R cuadrado

```
resumen_modelo <- summary(modelo_simple)

```

```r
r2 <- resumen_modelo$r.squared

print(r2)
```

Error frecuente

Un R^2 elevado no garantiza que un modelo sea adecuado para todas las finalidades, y un R^2 bajo no implica automáticamente que el modelo carezca de utilidad.

14.12. Visualizar la recta ajustada

Podemos representar el modelo sobre los datos:

Datos y regresión lineal

```r
ggplot(
    temp_humedad,
    aes(
        x = hr_media,
        y = temp_media
    )
) +
    geom_point() +
    geom_smooth(
        method = "lm",
        se = FALSE
    ) +
    labs(
        title = "Regresion lineal entre humedad y temperatura",
        x = "Humedad relativa media",
        y = "Temperatura media"
    ) +
    theme_minimal()
```

Idea clave

La recta representa el comportamiento estimado por el modelo. Los puntos muestran la variabilidad real que permanece alrededor de esa estimación.

14.13. Obtener predicciones

Podemos agregar a nuestra tabla los valores estimados por el modelo.

Agregar predicciones

```
temp_humedad <- temp_humedad %>%
    mutate(
        temp_predicha =
            predict(
                modelo_simple,
                newdata = temp_humedad
            )
    )

temp_humedad %>%
    select(
        temp_media,
        hr_media,
        temp_predicha
    ) %>%
    head()
```

Cómo leer este código

La función `predict` utiliza el modelo ajustado para calcular una predicción para cada observación.

14.14. Calcular residuos

El residuo es la diferencia entre el valor observado y el valor predicho:

$$e_i = y_i - \widehat{y}_i.$$

Podemos calcularlo:

Calcular residuos

```
temp_humedad <- temp_humedad %>%
    mutate(
        residuo =
            temp_media -
            temp_predicha
    )

temp_humedad %>%
    select(
        temp_media,
        temp_predicha,
        residuo
```

```
    ) %>%
    head()
```

Idea clave

Los residuos contienen información sobre aquello que el modelo no consiguió representar.

14.15. Visualizar residuos

Una visualización simple consiste en representar residuos frente a valores predichos.

Residuos y valores predichos

```
ggplot(
    temp_humedad,
    aes(
        x = temp_predicha,
        y = residuo
    )
) +
    geom_point() +
    geom_hline(
        yintercept = 0
    ) +
    labs(
        title = "Residuos del modelo",
        x = "Temperatura predicha",
        y = "Residuo"
    ) +
    theme_minimal()
```

Podemos observar:

- si los residuos aparecen aproximadamente distribuidos alrededor de cero;
- si existe algún patrón sistemático;
- si aparecen observaciones con errores especialmente grandes;
- si la dispersión cambia según el valor predicho.

Buena práctica

Si los residuos muestran una estructura clara, puede ser una señal de que el modelo lineal no está representando adecuadamente alguna característica de los datos.

14.16. Métricas básicas de error

Podemos resumir el tamaño de los errores mediante diferentes métricas.

Una de ellas es el error absoluto medio:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \widehat{y}_i|\,.$$

En R:

Calcular MAE

```
mae <- temp_humedad %>%
    summarise(
        MAE =
            mean(
                abs(residuo)
            )
    )

print(mae)
```

Otra posibilidad es la raíz del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2}.$$

Calcular RMSE

```
rmse <- temp_humedad %>%
    summarise(
        RMSE =
            sqrt(
                mean(
                    residuo^2
                )
            )
    )

print(rmse)
```

Cómo leer este código

El MAE expresa el tamaño medio del error utilizando valores absolutos.
El RMSE penaliza con mayor intensidad los errores grandes debido al uso del cuadrado.

Idea clave

Una métrica solamente tiene sentido cuando sabemos sobre qué datos fue calculada.

14.17. Evaluar con datos diferentes a los utilizados para ajustar

Hasta ahora ajustamos el modelo y calculamos sus errores sobre las mismas observaciones.

Esto puede dar una visión demasiado optimista de su capacidad predictiva.

Una estrategia sencilla consiste en separar los datos en:

- conjunto de entrenamiento;
- conjunto de evaluación.

Como estamos trabajando con datos temporales, podemos realizar una separación cronológica.

Ordenar los datos temporalmente

```
modelo_datos <- temp_humedad %>%
    arrange(Fecha)
```

Podemos utilizar, por ejemplo, aproximadamente el 80 % inicial para entrenamiento.

Definir punto de separación

```
n <- nrow(modelo_datos)

corte <- floor(
    0.8 * n
)
```

Luego:

Separar entrenamiento y evaluación

```
entrenamiento <- modelo_datos %>%
    slice(
        1:corte
```

```
    )

evaluacion <- modelo_datos %>%
    slice(
        (corte + 1):n
    )

cat(
    "Entrenamiento:",
    nrow(entrenamiento),
    "\n"
)

cat(
    "Evaluacion:",
    nrow(evaluacion),
    "\n"
)
```

Idea clave

Cuando existe un orden temporal y el objetivo futuro es realizar predicciones hacia adelante, una separación cronológica suele ser más apropiada que mezclar aleatoriamente observaciones pasadas y futuras.

14.18. Ajustar solamente con entrenamiento

Ahora ajustamos el modelo:

Modelo sobre entrenamiento

```
modelo_train <- lm(
    temp_media ~ hr_media,
    data = entrenamiento
)
```

Y predecimos sobre el conjunto de evaluación:

Predecir datos de evaluación

```
evaluacion <- evaluacion %>%
    mutate(
        temp_predicha =
            predict(
                modelo_train,
```

```
                newdata = evaluacion
            ),

        residuo =
            temp_media -
            temp_predicha
    )
```

Calculamos métricas:

Evaluar el modelo

```
metricas <- evaluacion %>%
    summarise(
        MAE =
            mean(
                abs(residuo)
            ),

        RMSE =
            sqrt(
                mean(
                    residuo^2
                )
            )
    )

print(metricas)
```

Buena práctica

Las métricas de evaluación deben calcularse, siempre que sea posible, sobre observaciones que no fueron utilizadas para ajustar el modelo.

14.19. Comparar observado y predicho

Podemos visualizar ambas series:

Preparar observado y predicho

```
comparacion <- evaluacion %>%
    select(
        Fecha,
        temp_media,
        temp_predicha
```

```
    ) %>%

    pivot_longer(
        cols = c(
            temp_media,
            temp_predicha
        ),

        names_to =
            "serie",

        values_to =
            "temperatura"
    )
```

Luego:

Valores observados y predichos

```
ggplot(
    comparacion,
    aes(
        x = Fecha,
        y = temperatura,
        color = serie
    )
) +
    geom_line() +
    labs(
        title = "Temperatura observada y predicha",
        x = "Fecha",
        y = "Temperatura",
        color = "Serie"
    ) +
    theme_minimal()
```

Idea clave

Visualizar observado y predicho permite detectar comportamientos que una única métrica puede ocultar.

14.20. Regresión lineal múltiple

La temperatura puede estar relacionada con más de una variable.

Podemos construir un modelo con varios predictores.

Primero preparamos observaciones completas:

Preparar varias variables

```
datos_multiple <- analisis %>%
    drop_na(
        temp_media,
        hr_media,
        rad_solar,
        precipitacion
    )
```

Luego ajustamos:

Regresión lineal múltiple

```
modelo_multiple <- lm(
    temp_media
        ~ hr_media
        + rad_solar
        + precipitacion,

    data =
        datos_multiple
)

summary(modelo_multiple)
```

El modelo tiene ahora una estructura general:

$$\widehat{y} = \widehat{\beta}_0 + \widehat{\beta}_1 x_1 + \widehat{\beta}_2 x_2 + \widehat{\beta}_3 x_3.$$

Cómo leer este código

Cada coeficiente representa la asociación estimada de una variable con la respuesta considerando simultáneamente las demás variables incluidas en el modelo.

Error frecuente

Interpretar cada coeficiente de una regresión múltiple como si hubiera sido calculado de forma aislada.

14.21. Relaciones entre variables explicativas

Antes de agregar muchas variables a un modelo también conviene observar cómo se relacionan entre ellas.

Correlaciones entre predictores

```
datos_multiple %>%
    select(
        hr_media,
        rad_solar,
        precipitacion
    ) %>%

    cor(
        use =
            "pairwise.complete.obs"
    )
```

Si dos variables contienen información muy similar, la interpretación de sus coeficientes puede volverse más difícil.

Idea clave

Agregar más variables no garantiza automáticamente un mejor modelo.

14.22. Comparar modelos

Podemos construir más de un modelo y comparar su desempeño sobre un conjunto de evaluación.

Por ejemplo:

Dos modelos candidatos

```
modelo_1 <- lm(
    temp_media ~ hr_media,
    data = entrenamiento
)

modelo_2 <- lm(
    temp_media
        ~ hr_media
        + rad_solar,
    data = entrenamiento
)
```

Para cada modelo podemos calcular predicciones sobre el mismo conjunto de evaluación y comparar métricas.

Buena práctica

Cuando comparamos modelos, debemos evaluarlos sobre el mismo conjunto de datos y utilizando las mismas métricas.

14.23. ¿Debemos mezclar todas las estaciones?

Una pregunta importante en nuestra base es si tiene sentido ajustar un único modelo utilizando todas las estaciones conjuntamente.

Las estaciones pueden presentar:

- condiciones ambientales diferentes;
- períodos diferentes de observación;
- disponibilidad diferente de sensores;
- patrones distintos de valores faltantes.

Podemos visualizar la relación por estación:

Relación por estación

```
ggplot(
    temp_humedad,
    aes(
        x = hr_media,
        y = temp_media
    )
) +
    geom_point() +
    geom_smooth(
        method = "lm",
        se = FALSE
    ) +
    facet_wrap(
        ~ estacion
    ) +
    theme_minimal()
```

Idea clave

Antes de combinar grupos diferentes dentro de un único modelo, conviene verificar si las relaciones observadas son razonablemente comparables.

14.24. Un flujo de regresión aplicada

Podemos resumir el procedimiento como:

Terminal

```
formular una pregunta
        |
        v
seleccionar variables
        |
        v
revisar calidad y faltantes
        |
        v
visualizar relaciones
        |
        v
calcular asociaciones
        |
        v
separar entrenamiento/evaluacion
        |
        v
ajustar modelo
        |
        v
predecir
        |
        v
calcular residuos y metricas
        |
        v
visualizar resultados
        |
        v
interpretar con cautela
```

Idea clave

El comando `lm` ocupa solamente una pequeña parte del flujo completo.

14.25. Actividad para probar

Para probar

Cada grupo deberá utilizar una base vinculada a su proyecto de aprendizaje automático y seleccionar una pregunta que pueda explorarse mediante regresión lineal.
La pregunta debe identificar:

- una variable respuesta numérica;
- al menos una posible variable explicativa numérica.

Para probar

El trabajo deberá incluir:

1. justificar las variables seleccionadas;
2. revisar valores faltantes;
3. indicar cuántas observaciones quedan disponibles para el análisis;
4. construir un gráfico de dispersión;
5. calcular la correlación entre las variables principales;
6. ajustar una regresión lineal simple;
7. interpretar el intercepto y la pendiente dentro del contexto del problema;
8. obtener valores predichos;
9. calcular residuos;
10. construir un gráfico de residuos;
11. calcular MAE y RMSE;
12. separar un conjunto de entrenamiento y otro de evaluación;
13. volver a ajustar el modelo utilizando solamente el conjunto de entrenamiento;
14. calcular las métricas sobre el conjunto de evaluación;
15. construir un gráfico que compare valores observados y predichos.

Para probar

Luego, construyan un segundo modelo incorporando al menos una variable explicativa adicional.
Comparen los dos modelos utilizando el mismo conjunto de evaluación.
Respondan:

1. ¿qué modelo obtuvo menor MAE?;
2. ¿qué modelo obtuvo menor RMSE?;
3. ¿agregar otra variable produjo una mejora relevante?;
4. ¿aparecen patrones en los residuos?;
5. ¿hay diferencias importantes entre grupos, estaciones, regiones o períodos?;
6. ¿qué limitaciones tendría este modelo si fuera utilizado para realizar predicciones reales?

Idea clave

El objetivo no es encontrar el modelo con más variables ni obtener una métrica perfecta. El objetivo es construir, evaluar e interpretar un procedimiento de análisis reproducible.

14.26. Cierre de la sección

En esta sección pasamos de la exploración visual de relaciones a la construcción de modelos de regresión lineal.

Utilizamos correlaciones y gráficos de dispersión para explorar asociaciones, ajustamos modelos simples y múltiples, generamos predicciones, estudiamos residuos y calculamos

métricas de error.

También introdujimos una idea fundamental: un modelo debe evaluarse sobre datos distintos de aquellos utilizados para ajustarlo.

La regresión trabaja a partir de una variable respuesta conocida que queremos describir o predecir.

En la próxima sección abordaremos una situación diferente: buscar estructuras dentro de los datos sin definir previamente una variable objetivo. Para ello introduciremos métodos de clustering, comenzando por el algoritmo `k-means`.

15. Clustering y segmentación exploratoria con k-means

15.1. Objetivo de la sección

En la sección anterior trabajamos con regresión lineal. Allí definimos una variable respuesta y utilizamos otras variables para intentar describirla o predecirla.

En esta sección abordaremos un problema diferente.

Supongamos que tenemos muchas observaciones y queremos responder:

> ¿Existen grupos de observaciones con características similares?

Pero ahora no tenemos una columna que indique previamente a qué grupo pertenece cada observación.

Este tipo de problema pertenece al aprendizaje no supervisado.

Trabajaremos con:

- selección de variables para clustering;
- tratamiento de valores faltantes;
- concepto de distancia;
- importancia de la escala de las variables;
- estandarización;
- algoritmo `k-means`;
- selección exploratoria del número de clusters;
- asignación de grupos a las observaciones;
- caracterización de los clusters;
- visualización e interpretación;
- limitaciones del procedimiento.

Idea clave

En clustering no indicamos previamente cuál es la respuesta correcta. El algoritmo intenta encontrar estructura a partir de las características de las observaciones.

15.2. Una pregunta exploratoria

Utilizaremos nuevamente la base meteorológica.

Podemos plantear una pregunta como:

> ¿Es posible identificar grupos de días con condiciones meteorológicas semejantes?

Por ejemplo, podríamos caracterizar cada día utilizando:

- temperatura media;
- humedad relativa media;
- radiación solar;
- precipitación;
- amplitud térmica.

No vamos a indicar previamente si un día es:

- cálido;
- frío;
- seco;
- húmedo;
- lluvioso.

Queremos observar si los datos producen agrupamientos que luego podamos interpretar.

Idea clave

Los clusters no tienen significado automático. Primero se calculan los grupos y después debemos estudiar qué características tienen.

15.3. Preparar los datos

Comenzamos cargando los paquetes y la base procesada.

Preparar el entorno

```
library(tidyverse)

datos <- read_csv(
    "data/processed/datos_meteorologicos_tidyverse.csv"
)

glimpse(datos)
```

Construimos una tabla con las variables que utilizaremos:

Seleccionar variables para clustering

```
datos_cluster <- datos %>%
    select(
        Fecha,
        estacion,
        temp_media,
        hr_media,
        rad_solar,
        precipitacion,
        amplitud_termica
    )

glimpse(datos_cluster)
```

Buena práctica

Las variables utilizadas para construir clusters deben seleccionarse porque tienen sentido para la pregunta planteada, no solamente porque están disponibles.

15.4. Comenzar con una estación

Como primera experiencia, puede ser conveniente trabajar con una sola estación.

Esto evita que diferencias estructurales entre estaciones dominen inicialmente el agrupamiento.

Seleccionar una estación

```
estacion_elegida <- "La Estanzuela"

datos_cluster <- datos_cluster %>%
    filter(
        estacion == estacion_elegida
    )

glimpse(datos_cluster)
```

Idea clave

Antes de agrupar observaciones procedentes de fuentes diferentes, conviene preguntarse si esas fuentes son realmente comparables.

15.5. Revisar valores faltantes

El algoritmo `k-means` no puede trabajar directamente con valores faltantes.

Primero revisamos su presencia:

Revisar faltantes

```
datos_cluster %>%
    summarise(
        across(
            everything(),
            ~ sum(is.na(.x))
        )
    )
```

Para este ejemplo utilizaremos observaciones completas en las variables elegidas:

Preparar registros completos

```
datos_cluster_completos <- datos_cluster %>%
    drop_na(
        temp_media,
        hr_media,
        rad_solar,
        precipitacion,
        amplitud_termica
    )

cat(
    "Registros originales:",
    nrow(datos_cluster),
    "\n"
)

cat(
    "Registros utilizados:",
    nrow(datos_cluster_completos),
    "\n"
)
```

Buena práctica

Eliminar observaciones faltantes puede cambiar la composición de los datos. Siempre debemos registrar cuántas observaciones quedan fuera del análisis.

15.6. Separar identificadores y variables de clustering

La fecha y el nombre de la estación son importantes para interpretar los resultados, pero no necesariamente deben utilizarse para calcular las distancias.

Construimos entonces una tabla solamente con las variables numéricas elegidas:

Matriz de variables

```
variables_cluster <- datos_cluster_completos %>%
    select(
        temp_media,
        hr_media,
        rad_solar,
        precipitacion,
        amplitud_termica
    )

glimpse(variables_cluster)
```

Idea clave

No todas las columnas de una base deben entrar en el clustering. Identificadores, códigos y fechas pueden ser fundamentales para interpretar las observaciones, pero no necesariamente para calcular semejanzas.

15.7. La idea de distancia

El clustering necesita alguna forma de cuantificar qué tan similares o diferentes son dos observaciones.

Una distancia frecuente es la distancia euclídea.

Para dos observaciones con variables $x_1, \ldots, x_p$:

$$d(x, y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}.$$

Si tenemos dos variables, podemos pensarla como la distancia ordinaria entre dos puntos del plano.

Idea clave

El resultado de un método basado en distancias depende directamente de las variables utilizadas y de sus escalas.

15.8. El problema de las escalas

Supongamos que utilizamos:

- temperatura, aproximadamente entre 10 y 35;

- humedad, aproximadamente entre 20 y 100;
- radiación solar, con valores de varios cientos;
- precipitación, frecuentemente próxima a cero pero ocasionalmente mucho mayor.

Si calculamos distancias directamente, las variables con mayor escala numérica pueden tener una influencia mucho mayor.

Idea clave

Una variable no debería ser más importante solamente porque está expresada con números mayores.

15.9. Estandarizar las variables

Una estrategia frecuente consiste en transformar cada variable mediante:

$$z = \frac{x - \bar{x}}{s},$$

donde:

- $\bar{x}$ es la media de la variable;
- s es su desviación estándar.

Después de esta transformación, cada variable tendrá aproximadamente:

$$\text{media} = 0$$

y

$$\text{desviación estándar} = 1.$$

En R podemos utilizar:

Estandarizar variables

```
variables_escaladas <- scale(
    variables_cluster
)
```

Podemos revisar:

Revisar variables estandarizadas

```
head(variables_escaladas)
```

Y comprobar sus medias:

Medias después de estandarizar

```
colMeans(
    variables_escaladas
)
```

También podemos revisar las desviaciones estándar:

Desviaciones estándar

```
apply(
    variables_escaladas,
    2,
    sd
)
```

Buena práctica

En métodos basados en distancias, siempre debemos revisar si las variables necesitan ser escaladas.

15.10. El algoritmo k-means

El algoritmo `k-means` busca dividir las observaciones en k grupos.

De forma simplificada, el procedimiento es:

1. elegir k centros iniciales;
2. asignar cada observación al centro más cercano;
3. recalcular el centro de cada grupo;
4. volver a asignar observaciones;
5. repetir hasta que las asignaciones se estabilicen.

Cada centro se denomina **centroide**.

Idea clave

El valor de k debe definirse antes de ejecutar `k-means`.

15.11. Primer modelo con k = 3

Comencemos suponiendo que queremos tres grupos.

Antes de ejecutar el algoritmo fijamos una semilla:

Fijar semilla

```
set.seed(123)
```

Luego:

Ajustar k-means

```
modelo_k3 <- kmeans(
    variables_escaladas,
    centers = 3,
    nstart = 25
)

print(modelo_k3)
```

Cómo leer este código

El argumento:

Terminal

```
  centers = 3
```

indica que queremos construir tres clusters.
El argumento:

Terminal

```
  nstart = 25
```

hace que el algoritmo pruebe diferentes configuraciones iniciales y conserve una solución apropiada entre ellas.

Buena práctica

Como `k-means` depende de centros iniciales, conviene utilizar múltiples inicializaciones y fijar una semilla para obtener resultados reproducibles.

15.12. Agregar el cluster a la base

El objeto generado por `kmeans` contiene la asignación de cada observación:

Ver primeras asignaciones

```
head(
    modelo_k3$cluster
)
```

Podemos agregar esa información a nuestra tabla original:

Agregar clusters

```
resultado_cluster <- datos_cluster_completos %>%
    mutate(
        cluster =
            factor(
                modelo_k3$cluster
            )
    )

resultado_cluster %>%
    select(
        Fecha,
        temp_media,
        hr_media,
        rad_solar,
        precipitacion,
        amplitud_termica,
        cluster
    ) %>%
    head()
```

Idea clave

Los números 1, 2 y 3 no tienen un significado natural. Son solamente identificadores de los grupos encontrados.

15.13. ¿Cuántas observaciones tiene cada cluster?

Podemos comenzar contando:

Tamaño de los clusters

```
resultado_cluster %>%
    count(
        cluster
    )
```

Esto permite detectar, por ejemplo, si un cluster contiene solamente unas pocas observaciones.

Buena práctica

El tamaño de los clusters forma parte de la interpretación. Un grupo extremadamente pequeño puede representar un comportamiento particular, observaciones extremas o una solución poco estable.

15.14. Caracterizar los clusters

Ahora podemos estudiar las características promedio de cada grupo.

Resumen por cluster

```
perfil_clusters <- resultado_cluster %>%
    group_by(
        cluster
    ) %>%

    summarise(
        registros = n(),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        hr_media =
            mean(
                hr_media,
                na.rm = TRUE
            ),

        rad_solar =
```

```
            mean(
                rad_solar,
                na.rm = TRUE
            ),

        precipitacion =
            mean(
                precipitacion,
                na.rm = TRUE
            ),

        amplitud_termica =
            mean(
                amplitud_termica,
                na.rm = TRUE
            ),

        .groups = "drop"
    )

print(perfil_clusters)
```

Idea clave

La interpretación comienza después del clustering. Debemos observar qué distingue a cada grupo.

A partir de este resumen podríamos encontrar, por ejemplo, un grupo caracterizado por:

- mayor temperatura;
- menor humedad;
- mayor radiación;

y otro con:

- mayor humedad;
- menor radiación;
- mayor precipitación.

Pero esas interpretaciones deben surgir de los resultados y no definirse de antemano.

15.15. Examinar los centroides estandarizados

El objeto del modelo contiene los centroides:

Centroides

```
modelo_k3$centers
```

Como utilizamos variables estandarizadas:

- valores positivos indican valores por encima de la media;
- valores negativos indican valores por debajo de la media.

Por ejemplo, un centroide podría mostrar:

Terminal

```
temp_media          0.8
hr_media           -0.7
rad_solar           0.9
precipitacion      -0.4
amplitud_termica    0.5
```

Una posible lectura sería:

> Este cluster presenta, en promedio, temperaturas y radiación superiores a la media y humedad inferior a la media.

Idea clave

Trabajar con centroides estandarizados facilita comparar variables que originalmente utilizaban escalas diferentes.

15.16. Visualizar los clusters

Aunque el algoritmo utilice cinco variables, podemos observar proyecciones utilizando dos variables.

Por ejemplo:

Temperatura y humedad por cluster

```
ggplot(
    resultado_cluster,
    aes(
        x = temp_media,
        y = hr_media,
        color = cluster
    )
) +
```

```
    geom_point() +
    labs(
        title = "Clusters de condiciones meteorologicas",
        subtitle = "Proyeccion sobre temperatura y humedad",
        x = "Temperatura media",
        y = "Humedad relativa media",
        color = "Cluster"
    ) +
    theme_minimal()
```

Podemos construir otra visualización:

Radiación y precipitación por cluster

```
ggplot(
    resultado_cluster,
    aes(
        x = rad_solar,
        y = precipitacion,
        color = cluster
    )
) +
    geom_point() +
    labs(
        title = "Clusters de condiciones meteorologicas",
        subtitle = "Proyeccion sobre radiacion y precipitacion",
        x = "Radiacion solar",
        y = "Precipitacion",
        color = "Cluster"
    ) +
    theme_minimal()
```

Error frecuente

Si dos clusters aparecen mezclados en un gráfico de dos variables, eso no significa necesariamente que el algoritmo no haya encontrado diferencias. El clustering fue realizado utilizando todas las variables seleccionadas.

15.17. Observar los clusters en el tiempo

Como conservamos la fecha original, podemos estudiar cuándo aparecen los diferentes grupos.

Cluster a lo largo del tiempo

```
ggplot(
    resultado_cluster,
    aes(
        x = Fecha,
        y = temp_media,
        color = cluster
    )
) +
    geom_point() +
    labs(
        title = "Clusters a lo largo del tiempo",
        x = "Fecha",
        y = "Temperatura media",
        color = "Cluster"
    ) +
    theme_minimal()
```

Esto puede sugerir preguntas como:

- ¿algunos clusters aparecen principalmente en determinadas estaciones del año?;
- ¿existen períodos dominados por un tipo de comportamiento?;
- ¿los clusters representan principalmente diferencias estacionales?;

Idea clave

Una vez obtenidos los clusters, podemos relacionarlos nuevamente con variables que no fueron utilizadas para construirlos.

15.18. ¿Cómo elegir el número de clusters?

Hasta ahora utilizamos $k = 3$ arbitrariamente.

Podemos comparar diferentes valores de k.

Una medida disponible en el resultado de `kmeans` es:

Terminal

```
tot.withinss
```

que representa la variabilidad interna total de los clusters.

Cuanto menor sea este valor, más próximas están las observaciones a sus respectivos centroides.

Comparar diferentes valores de k

```
resultados_k <- tibble(
    k = 1:10,
    withinss = NA_real_
)

for (i in 1:10) {

    set.seed(123)

    modelo <- kmeans(
        variables_escaladas,
        centers = i,
        nstart = 25
    )

    resultados_k$withinss[i] <-
        modelo$tot.withinss
}

print(resultados_k)
```

Podemos graficar:

Método del codo

```
ggplot(
    resultados_k,
    aes(
        x = k,
        y = withinss
    )
) +
    geom_line() +
    geom_point() +
    scale_x_continuous(
        breaks = 1:10
    ) +
    labs(
        title = "Variabilidad interna segun cantidad de clusters",
        x = "Cantidad de clusters",
        y = "Variabilidad interna total"
    ) +
    theme_minimal()
```

Cómo leer este código

Al aumentar k, la variabilidad interna siempre disminuye.
Por eso no buscamos simplemente el menor valor.
Buscamos un punto a partir del cual agregar nuevos clusters produce mejoras cada vez menores.

Idea clave

El método del codo es una herramienta exploratoria. No produce necesariamente una única respuesta correcta para k.

15.19. Otra medida: silhouette

También podemos estudiar qué tan bien queda asignada cada observación a su cluster.

Una herramienta frecuente es la medida de *silhouette*.

Para utilizarla cargamos:

Cargar paquete cluster

```
library(cluster)
```

Para nuestro modelo:

Calcular silhouette

```
distancias <- dist(
    variables_escaladas
)

silhouette_k3 <- silhouette(
    modelo_k3$cluster,
    distancias
)

summary(
    silhouette_k3
)
```

Podemos obtener el promedio:

Silhouette promedio

```
media_silhouette <- mean(
    silhouette_k3[, "sil_width"]
)

print(
    media_silhouette
)
```

De forma general:

- valores mayores indican una separación más clara;
- valores próximos a cero indican observaciones cercanas a fronteras entre grupos;
- valores negativos pueden indicar observaciones posiblemente mejor asociadas a otro cluster.

Buena práctica

La selección de k puede combinar diferentes evidencias: métricas, visualizaciones, estabilidad e interpretación del problema.

15.20. Comparar silhouette para varios valores de k

Podemos construir una función:

Función para silhouette promedio

```
calcular_silhouette <- function(
    k,
    datos_escalados
) {

    set.seed(123)

    modelo <- kmeans(
        datos_escalados,
        centers = k,
        nstart = 25
    )

    sil <- silhouette(
        modelo$cluster,
        dist(datos_escalados)
    )
```

```
18
19      return(
20          mean(
21              sil[, "sil_width"]
22          )
23      )
24  }
```

Luego:

Evaluar distintos valores de k

```
comparacion_k <- tibble(
    k = 2:8
) %>%

    mutate(
        silhouette =
            map_dbl(
                k,
                ~ calcular_silhouette(
                    .x,
                    variables_escaladas
                )
            )
    )

print(comparacion_k)
```

Y podemos visualizar:

Silhouette según k

```
ggplot(
    comparacion_k,
    aes(
        x = k,
        y = silhouette
    )
) +
    geom_line() +
    geom_point() +
    scale_x_continuous(
        breaks = 2:8
    ) +
```

```
    labs(
        title = "Silhouette promedio segun cantidad de clusters",
        x = "Cantidad de clusters",
        y = "Silhouette promedio"
    ) +
    theme_minimal()
```

15.21. Valores extremos y clustering

Los valores extremos pueden afectar significativamente los centroides de `k-means`.

Por ejemplo, unos pocos días con precipitaciones excepcionalmente elevadas podrían modificar la estructura de los clusters.

Podemos revisar:

Examinar valores extremos

```
resultado_cluster %>%
    arrange(
        desc(precipitacion)
    ) %>%

    select(
        Fecha,
        precipitacion,
        temp_media,
        hr_media,
        cluster
    ) %>%

    slice_head(
        n = 10
    )
```

Idea clave

Detectar una observación extrema no significa que debamos eliminarla. En meteorología, producción, economía y muchas otras áreas, los eventos extremos pueden ser precisamente las observaciones más interesantes.

15.22. Variables con distribuciones muy asimétricas

La precipitación suele presentar una característica particular:

- muchos días con valor cero;

- algunos días con valores pequeños;
- pocos días con valores muy elevados.

Podemos observar su distribución:

Distribución de precipitación

```
ggplot(
    datos_cluster_completos,
    aes(
        x = precipitacion
    )
) +
    geom_histogram(
        bins = 30
    ) +
    labs(
        title = "Distribucion de precipitacion",
        x = "Precipitacion",
        y = "Frecuencia"
    ) +
    theme_minimal()
```

En algunos problemas podría resultar útil estudiar una transformación como:

$$x' = \log(1 + x).$$

En R:

Transformar precipitación

```
datos_cluster_completos <- datos_cluster_completos %>%
    mutate(
        log_precipitacion =
            log1p(
                precipitacion
            )
    )
```

Buena práctica

Las transformaciones deben justificarse a partir de la distribución y del significado de la variable. No deben aplicarse automáticamente.

15.23. Nombrar los clusters

Después de caracterizar los grupos, podemos asignar nombres descriptivos.

Supongamos que el análisis muestra claramente tres perfiles.

Podríamos construir:

Asignar nombres interpretativos

```
resultado_cluster <- resultado_cluster %>%
    mutate(
        tipo_dia = case_when(
            cluster == "1"
                ~ "perfil A",

            cluster == "2"
                ~ "perfil B",

            cluster == "3"
                ~ "perfil C"
        )
    )
```

Después de analizar los centroides podríamos reemplazar esos nombres por etiquetas más informativas.

Error frecuente

Asignar nombres como “día cálido” o “día lluvioso” antes de revisar las características reales de los clusters.

Idea clave

Primero agrupamos. Después caracterizamos. Finalmente interpretamos y, si corresponde, asignamos nombres.

15.24. ¿Qué ocurre si incluimos varias estaciones?

Una extensión natural consiste en repetir el análisis utilizando varias estaciones.

Sin embargo, debemos preguntarnos:

- ¿todas tienen las mismas variables disponibles?;
- ¿los períodos observados son comparables?;
- ¿las diferencias entre estaciones dominarán los clusters?;
- ¿queremos agrupar días o queremos indirectamente agrupar estaciones?;

Después de construir los clusters podemos estudiar:

Clusters por estación

```
resultado_varias_estaciones %>%
    count(
        estacion,
        cluster
    ) %>%

    group_by(
        estacion
    ) %>%

    mutate(
        proporcion =
            n / sum(n)
    )
```

Idea clave

Si casi todas las observaciones de una estación caen en un único cluster, puede ser necesario investigar si el algoritmo está encontrando tipos de días o simplemente diferencias sistemáticas entre estaciones.

15.25. Regresión y clustering responden preguntas diferentes

En la sección anterior trabajamos con una estructura como:

Terminal

```
variables explicativas
        |
        v
variable respuesta conocida
        |
        v
regresion
```

En clustering trabajamos con:

Terminal

```
variables descriptivas
        |
        v
```

```
similitud entre observaciones
        |
        v
grupos no definidos previamente
```

Podemos resumir:

	Regresión	**Clustering**
Variable objetivo	Sí	No
Tipo	Supervisado	No supervisado
Pregunta típica	¿Qué valor esperamos?	¿Qué observaciones son similares?
Resultado	Predicción o relación	Asignación a grupos
Ejemplo	Predecir temperatura	Identificar tipos de días

15.26. Un flujo de clustering aplicado

Podemos resumir el procedimiento:

Terminal

```
formular una pregunta
        |
        v
seleccionar variables
        |
        v
revisar calidad y faltantes
        |
        v
estudiar distribuciones
        |
        v
transformar si corresponde
        |
        v
estandarizar
        |
        v
probar diferentes valores de k
        |
        v
ajustar k-means
        |
        v
```

```
caracterizar clusters
        |
        v
visualizar
        |
        v
relacionar con otras variables
        |
        v
interpretar
```

Idea clave

Ejecutar `kmeans()` es solamente una pequeña parte del análisis.

15.27. Actividad para probar

Para probar

Cada grupo deberá utilizar una base vinculada a su proyecto y formular una pregunta exploratoria que pueda abordarse mediante clustering.
El objetivo debe estar expresado en términos de similitud entre observaciones y no debe utilizar una variable objetivo conocida para definir los grupos.

Para probar

El trabajo deberá incluir:

1. definir qué representa una observación que será agrupada;
2. seleccionar entre tres y seis variables numéricas relevantes;
3. justificar por qué esas variables fueron elegidas;
4. revisar valores faltantes;
5. revisar las distribuciones de las variables;
6. identificar posibles valores extremos;
7. decidir si alguna variable requiere una transformación;
8. estandarizar las variables utilizadas;
9. aplicar `k-means`;
10. probar diferentes valores de k;
11. construir el gráfico del método del codo;
12. utilizar al menos otra evidencia para discutir la elección de k, como silhouette o interpretación de los grupos;
13. agregar la asignación de cluster a la base original;
14. indicar cuántas observaciones pertenecen a cada cluster;
15. construir una tabla que caracterice los clusters mediante sus variables originales;
16. construir al menos dos visualizaciones de los grupos;

17. proponer una interpretación para cada cluster.

Para probar

Respondan además:

1. ¿qué significado práctico parece tener cada cluster?;
2. ¿algún cluster está asociado a un período, región, estación, categoría u otra característica que no fue utilizada directamente en el algoritmo?;
3. ¿los clusters son claramente diferentes o existen muchas observaciones próximas a las fronteras?;
4. ¿qué efecto creen que tendría no estandarizar las variables?;
5. ¿existen observaciones extremas que influyan fuertemente en los resultados?;
6. ¿cambiar el valor de k altera sustancialmente la interpretación?;
7. ¿los grupos encontrados resultan útiles para comprender el problema del proyecto?

Idea clave

Un clustering útil no es necesariamente aquel que produce grupos perfectamente separados. Debe producir una estructura que podamos describir, investigar y relacionar con la pregunta del problema.

15.28. Cierre de la sección

En esta sección introdujimos el clustering como herramienta de análisis exploratorio no supervisado.

Aprendimos que los métodos basados en distancias requieren especial atención a las escalas de las variables y utilizamos estandarización para construir una representación comparable.

Aplicamos `k-means`, estudiamos diferentes valores de k, utilizamos variabilidad interna y silhouette como herramientas de apoyo y caracterizamos los grupos obtenidos utilizando las variables originales.

También vimos que los clusters no contienen una interpretación automática: debemos volver a los datos y estudiar qué diferencia a cada grupo.

Con regresión y clustering incorporamos dos formas diferentes de análisis multivariable. En la próxima sección volveremos a la dimensión temporal de los datos para estudiar agregaciones, tendencias, cambios y comparaciones a lo largo del tiempo.

16. Análisis temporal, agregaciones y tendencias

16.1. Objetivo de la sección

Muchas bases utilizadas en ciencia de datos contienen una dimensión temporal.

Una observación puede representar:

- una medición meteorológica diaria;
- la producción mensual de un establecimiento;
- una transacción;
- una lectura de un sensor;
- una observación anual sobre uso del suelo;
- un registro realizado en una fecha y hora determinada.

Hasta ahora utilizamos fechas principalmente para ordenar, filtrar y visualizar observaciones.

En esta sección avanzaremos hacia el análisis de la evolución de las variables a lo largo del tiempo.

Trabajaremos con:

- extracción de componentes temporales;
- cambio de granularidad temporal;
- agregaciones mensuales y anuales;
- comparación entre períodos;
- funciones `lag` y `lead`;
- diferencias y tasas de variación;
- acumulados;
- medias móviles;
- tendencias;
- comparación entre grupos;
- períodos incompletos y observaciones faltantes.

Idea clave

Trabajar con tiempo no consiste solamente en ordenar una tabla por fecha. También debemos decidir qué escala temporal es apropiada para la pregunta que queremos responder.

16.2. Preparar los datos

Utilizaremos nuevamente la base meteorológica procesada.

Preparar el entorno

```
library(tidyverse)
library(lubridate)
library(slider)

datos <- read_csv(
    "data/processed/datos_meteorologicos_tidyverse.csv"
)

glimpse(datos)
```

Verificamos que la columna de fecha tenga el tipo correcto:

Verificar la variable temporal

```
class(
    datos$Fecha
)
```

Si fuera necesario:

Convertir a fecha

```
datos <- datos %>%
    mutate(
        Fecha = as.Date(Fecha)
    )
```

Buena práctica

Antes de realizar cualquier operación temporal, conviene verificar que la fecha esté almacenada realmente como una variable temporal y no como texto.

16.3. Extraer componentes de una fecha

El paquete `lubridate` permite obtener diferentes componentes temporales.

Por ejemplo:

Extraer componentes temporales

```
datos <- datos %>%
    mutate(
        anio = year(Fecha),
        mes = month(Fecha),
        dia = day(Fecha),
        dia_anio = yday(Fecha),
        semana = isoweek(Fecha),
        dia_semana = wday(
            Fecha,
            label = TRUE
        )
    )

datos %>%
    select(
        Fecha,
        anio,
        mes,
        dia,
        dia_anio,
        semana,
        dia_semana
    ) %>%
    head()
```

Cómo leer este código

A partir de una única variable de fecha podemos construir diferentes variables temporales.
La elección depende de la pregunta de análisis.

Idea clave

No siempre necesitamos guardar año, mes y día como columnas permanentes. Podemos generarlos cuando sean necesarios para una operación específica.

16.4. Cambiar la granularidad temporal

Supongamos que tenemos observaciones diarias, pero queremos estudiar comportamiento mensual.

Podemos crear una variable que represente el mes:

Construir una referencia mensual

```
datos <- datos %>%
    mutate(
        mes_fecha = floor_date(
            Fecha,
            unit = "month"
        )
    )

datos %>%
    select(
        Fecha,
        mes_fecha
    ) %>%
    head()
```

La función `floor_date` lleva cada fecha al comienzo del período correspondiente.

Por ejemplo:

Terminal

```
2016-01-03 -> 2016-01-01
2016-01-18 -> 2016-01-01
2016-02-14 -> 2016-02-01
```

Podemos hacer lo mismo para años:

Referencia anual

```
datos <- datos %>%
    mutate(
        anio_fecha = floor_date(
            Fecha,
            unit = "year"
        )
    )
```

16.5. De datos diarios a datos mensuales

Supongamos que queremos construir un resumen mensual para cada estación.

No todas las variables deben agregarse de la misma forma.

Por ejemplo:

- la temperatura media puede resumirse mediante una media;
- la precipitación acumulada diaria puede sumarse;
- una temperatura máxima puede resumirse mediante el máximo;
- la cantidad de observaciones puede contarse.

Idea clave

Cambiar la escala temporal exige pensar qué operación tiene sentido para cada variable.

Podemos construir:

Resumen mensual por estación

```
mensual <- datos %>%
    mutate(
        mes = floor_date(
            Fecha,
            "month"
        )
    ) %>%

    group_by(
        estacion,
        mes
    ) %>%

    summarise(
        registros = n(),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        temp_max =
            max(
                temp_max,
                na.rm = TRUE
            ),

        temp_min =
            min(
                temp_min,
                na.rm = TRUE
            ),
```

```
34
35          precipitacion =
36              sum(
37                  precipitacion,
38                  na.rm = TRUE
39              ),
40
41          hr_media =
42              mean(
43                  hr_media,
44                  na.rm = TRUE
45              ),
46
47          rad_solar =
48              mean(
49                  rad_solar,
50                  na.rm = TRUE
51              ),
52
53          .groups = "drop"
54      )
55
56  glimpse(mensual)
```

16.6. Una precaución con na.rm

Consideremos:

Código R

```
1  sum(
2      precipitacion,
3      na.rm = TRUE
4  )
```

Si todos los valores de un mes fueran `NA`, esta expresión podría producir un resultado que no representa adecuadamente la ausencia completa de información.

Una alternativa más cuidadosa es:

Suma conservando ausencia total

```
1  sumar_si_hay_datos <- function(x) {
2
3      if (
```

```
        all(is.na(x))
    ) {
        return(NA_real_)
    }

    sum(
        x,
        na.rm = TRUE
    )
}
```

Ahora podemos usar:

Precipitación mensual

```
mensual <- datos %>%
    mutate(
        mes = floor_date(
            Fecha,
            "month"
        )
    ) %>%

    group_by(
        estacion,
        mes
    ) %>%

    summarise(
        precipitacion =
            sumar_si_hay_datos(
                precipitacion
            ),

        .groups = "drop"
    )
```

Idea clave

Ignorar valores faltantes no debe transformar accidentalmente “no tenemos información” en un valor válido.

16.7. Registrar cuántas observaciones participan

Además del valor resumido, puede ser importante conocer cuántas observaciones fueron utilizadas.

Cantidad de observaciones disponibles

```
mensual <- datos %>%
    mutate(
        mes = floor_date(
            Fecha,
            "month"
        )
    ) %>%

    group_by(
        estacion,
        mes
    ) %>%

    summarise(
        dias_registrados =
            n(),

        dias_temp_disponible =
            sum(
                !is.na(temp_media)
            ),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        .groups = "drop"
    )
```

Buena práctica

Una media mensual calculada con 30 observaciones y otra calculada con 4 observaciones no deberían interpretarse sin conocer esa diferencia.

16.8. ¿Está completo el período?

Podemos comparar la cantidad de días disponibles con la cantidad esperada en cada mes.

Cantidad de días esperados

```
mensual <- mensual %>%
    mutate(
```

```
        dias_esperados =
            days_in_month(
                mes
            ),

        porcentaje_dias =
            dias_temp_disponible /
            dias_esperados *
            100
    )
```

Ahora podemos encontrar períodos con baja cobertura:

Buscar meses incompletos

```
mensual %>%
    filter(
        porcentaje_dias < 80
    ) %>%
    arrange(
        porcentaje_dias
    )
```

Idea clave

Antes de comparar períodos agregados debemos verificar si representan cantidades semejantes de información.

16.9. Construir resúmenes anuales

El mismo principio puede aplicarse a una escala anual.

Resumen anual

```
anual <- datos %>%
    mutate(
        anio = year(Fecha)
    ) %>%

    group_by(
        estacion,
        anio
    ) %>%

    summarise(
```

```
        registros = n(),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        precipitacion =
            sumar_si_hay_datos(
                precipitacion
            ),

        hr_media =
            mean(
                hr_media,
                na.rm = TRUE
            ),

        .groups = "drop"
    )

print(anual)
```

16.10. Visualizar una serie agregada

Podemos representar la temperatura media mensual:

Temperatura mensual

```
ggplot(
    mensual,
    aes(
        x = mes,
        y = temp_media,
        color = estacion
    )
) +
    geom_line() +
    labs(
        title = "Temperatura media mensual",
        x = "Mes",
        y = "Temperatura media",
        color = "Estacion"
    ) +
    theme_minimal()
```

O separar mediante paneles:

Temperatura mensual por estación

```
ggplot(
    mensual,
    aes(
        x = mes,
        y = temp_media
    )
) +
    geom_line() +
    facet_wrap(
        ~ estacion
    ) +
    labs(
        title = "Temperatura media mensual por estacion",
        x = "Mes",
        y = "Temperatura media"
    ) +
    theme_minimal()
```

16.11. Comparar una observación con la anterior

Una operación muy frecuente consiste en comparar un valor con el período anterior.

La función `lag` permite recuperar el valor anterior.

Primero ordenamos correctamente los datos:

Preparar comparación temporal

```
mensual <- mensual %>%
    group_by(
        estacion
    ) %>%
    arrange(
        estacion,
        mes
    ) %>%

    mutate(
        temp_anterior =
            lag(
```

```
                temp_media
            )
    ) %>%

    ungroup()
```

Podemos calcular la diferencia:

Variación respecto al mes anterior

```
mensual <- mensual %>%
    group_by(
        estacion
    ) %>%
    arrange(
        mes,
        .by_group = TRUE
    ) %>%

    mutate(
        variacion_temp =
            temp_media -
            lag(temp_media)
    ) %>%

    ungroup()
```

Si:

$$T_t$$

representa la temperatura del período actual y:

$$T_{t-1}$$

la del período anterior, entonces:

$$\Delta T_t = T_t - T_{t-1}.$$

Idea clave

`lag` depende del orden de las observaciones. Antes de usarlo debemos ordenar correctamente los datos dentro de cada grupo.

16.12. Mirar hacia el período siguiente

La función `lead` hace la operación opuesta.

Valor del período siguiente

```
mensual <- mensual %>%
    group_by(
        estacion
    ) %>%

    arrange(
        mes,
        .by_group = TRUE
    ) %>%

    mutate(
        temp_siguiente =
            lead(
                temp_media
            )
    ) %>%

    ungroup()
```

Esto puede resultar útil, por ejemplo, para construir posteriormente problemas predictivos donde una observación del presente se relacione con una variable futura.

Buena práctica

Al utilizar `lead` para construir una variable objetivo futura, debemos prestar especial atención a no utilizar accidentalmente información futura como predictor.

16.13. Tasa de variación

Para algunas variables puede ser interesante calcular una variación relativa.

Si una variable toma valores x_{t-1} y x_t:

$$r_t = \frac{x_t - x_{t-1}}{x_{t-1}} \times 100.$$

En R:

Tasa de variación

```
mensual <- mensual %>%
    group_by(
        estacion
    ) %>%

    arrange(
        mes,
        .by_group = TRUE
    ) %>%

    mutate(
        precipitacion_anterior =
            lag(
                precipitacion
            ),

        variacion_precipitacion_pct =
            if_else(
                !is.na(
                    precipitacion_anterior
                ) &
                precipitacion_anterior != 0,

                (
                    precipitacion -
                    precipitacion_anterior
                ) /
                precipitacion_anterior *
                100,

                NA_real_
            )
    ) %>%

    ungroup()
```

Buena práctica

Una tasa relativa requiere atención especial cuando el valor anterior es cero o está ausente.

Idea clave

No todas las variables se interpretan bien mediante porcentajes de variación. La transformación debe tener sentido dentro del problema.

16.14. Valores acumulados

Otra operación temporal frecuente es construir acumulados.

Por ejemplo, precipitación acumulada dentro de cada año.

Primero construimos datos diarios:

Precipitación acumulada anual

```
precipitacion_diaria <- datos %>%
    select(
        Fecha,
        estacion,
        precipitacion
    ) %>%

    mutate(
        anio = year(Fecha)
    ) %>%

    group_by(
        estacion,
        anio
    ) %>%

    arrange(
        Fecha,
        .by_group = TRUE
    ) %>%

    mutate(
        precipitacion_acumulada =
            cumsum(
                replace_na(
                    precipitacion,
                    0
                )
            )
    ) %>%

    ungroup()
```

Error frecuente

Reemplazar automáticamente una precipitación faltante por cero implica asumir que no llovió. Esa decisión solamente es válida si el significado del dato permite justificarla.

Una alternativa más prudente puede ser calcular el acumulado solamente después de haber establecido una política explícita para los valores faltantes.

16.15. Visualizar un acumulado

Podemos seleccionar una estación y un año:

Seleccionar una serie acumulada

```
acumulado_2016 <- precipitacion_diaria %>%
    filter(
        estacion == "La Estanzuela",
        anio == 2016
    )
```

Luego:

Precipitación acumulada

```
ggplot(
    acumulado_2016,
    aes(
        x = Fecha,
        y = precipitacion_acumulada
    )
) +
    geom_line() +
    labs(
        title = "Precipitacion acumulada",
        subtitle = "La Estanzuela - 2016",
        x = "Fecha",
        y = "Precipitacion acumulada"
    ) +
    theme_minimal()
```

16.16. Medias móviles

Los datos temporales pueden presentar mucha variación entre observaciones consecutivas.

Una media móvil permite construir una versión suavizada de la serie.

Para una ventana de tamaño k:

$$M_t = \frac{x_t + x_{t-1} + \cdots + x_{t-k+1}}{k}.$$

Utilizaremos el paquete `slider`.

Supongamos que queremos una media móvil de tres meses:

Media móvil de tres períodos

```
mensual <- mensual %>%
    group_by(
        estacion
    ) %>%

    arrange(
        mes,
        .by_group = TRUE
    ) %>%

    mutate(
        media_movil_3 =
            slide_dbl(
                temp_media,
                mean,
                .before = 2,
                .complete = TRUE
            )
    ) %>%

    ungroup()
```

Cómo leer este código

En cada posición utilizamos:

- el valor actual;
- los dos valores anteriores.

Por eso la ventana contiene tres observaciones.
El argumento:

Terminal

```
.complete = TRUE
```

hace que solamente se calcule el resultado cuando la ventana está completa.

16.17. Comparar la serie original y la media móvil

Seleccionamos una estación:

Preparar serie

```
estacion_mensual <- mensual %>%
    filter(
        estacion ==
            "La Estanzuela"
    )
```

Transformamos a formato largo:

Preparar serie original y suavizada

```
comparacion_media <- estacion_mensual %>%
    select(
        mes,
        temp_media,
        media_movil_3
    ) %>%

    pivot_longer(
        cols = c(
            temp_media,
            media_movil_3
        ),

        names_to = "serie",
        values_to = "temperatura"
    )
```

Y graficamos:

Serie y media móvil

```
ggplot(
    comparacion_media,
    aes(
        x = mes,
        y = temperatura,
        color = serie
    )
) +
    geom_line() +
    labs(
        title = "Temperatura mensual y media movil",
        x = "Mes",
```

```
        y = "Temperatura",
        color = "Serie"
    ) +
    theme_minimal()
```

Idea clave

Una media móvil suaviza variaciones de corto plazo, pero también elimina parte de la información original.

16.18. Ventanas por cantidad de observaciones y ventanas temporales

Una ventana de tres observaciones no significa necesariamente tres meses si existen períodos ausentes.

Por ejemplo, una serie podría contener:

Terminal

```
enero
febrero
abril
```

y una ventana de tres observaciones incluiría esos tres registros, aunque marzo esté ausente.

Idea clave

En análisis temporal debemos distinguir entre una secuencia de observaciones y una secuencia regular de períodos.

16.19. Detectar períodos ausentes

Podemos verificar si existen meses faltantes usando `complete`.

Por ejemplo:

Completar secuencia mensual

```
mensual_completo <- mensual %>%
    group_by(
        estacion
    ) %>%

    complete(
```

```
        mes = seq.Date(
            min(mes),
            max(mes),
            by = "month"
        )
    ) %>%

    ungroup()
```

Los períodos que no existían aparecerán como nuevas filas con valores faltantes.

Buena práctica

Antes de aplicar operaciones secuenciales como `lag` o medias móviles, conviene verificar si la serie contiene períodos ausentes.

16.20. Comparar meses entre diferentes años

Otra pregunta posible es:

> ¿Los meses de enero presentan comportamientos semejantes entre diferentes años?

Podemos construir:

Mes y año

```
mensual <- mensual %>%
    mutate(
        anio =
            year(mes),

        numero_mes =
            month(mes),

        nombre_mes =
            month(
                mes,
                label = TRUE,
                abbr = TRUE
            )
    )
```

Luego podemos comparar:

Temperatura por mes y año

```
mensual %>%
    filter(
        estacion ==
            "La Estanzuela"
    ) %>%

    ggplot(
        aes(
            x = numero_mes,
            y = temp_media,
            group = anio,
            color = factor(anio)
        )
    ) +
    geom_line() +
    geom_point() +
    scale_x_continuous(
        breaks = 1:12
    ) +
    labs(
        title =
            "Temperatura mensual por ano",

        x =
            "Mes",

        y =
            "Temperatura media",

        color =
            "Ano"
    ) +
    theme_minimal()
```

Idea clave

Comparar la misma posición del calendario entre diferentes años ayuda a distinguir patrones estacionales de cambios de largo plazo.

16.21. Construir un perfil mensual

Podemos resumir todos los años según el mes del calendario:

Perfil mensual

```
perfil_mensual <- mensual %>%
    mutate(
        numero_mes =
            month(mes)
    ) %>%

    group_by(
        estacion,
        numero_mes
    ) %>%

    summarise(
        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        precipitacion =
            mean(
                precipitacion,
                na.rm = TRUE
            ),

        .groups = "drop"
    )
```

Visualizamos:

Patrón mensual promedio

```
ggplot(
    perfil_mensual,
    aes(
        x = numero_mes,
        y = temp_media,
        color = estacion
    )
) +
    geom_line() +
    geom_point() +
    scale_x_continuous(
        breaks = 1:12
    ) +
    labs(
```

```
        title =
            "Perfil mensual promedio de temperatura",

        x =
            "Mes",

        y =
            "Temperatura media",

        color =
            "Estacion"
    ) +
    theme_minimal()
```

Idea clave

Un patrón que se repite según el calendario puede sugerir estacionalidad, pero aquí utilizaremos el término de forma descriptiva y no desarrollaremos todavía modelos formales de series temporales.

16.22. Explorar una tendencia de largo plazo

Podemos utilizar una regresión lineal como herramienta descriptiva de tendencia.

Supongamos que tenemos medias anuales:

Preparar datos anuales

```
tendencia_anual <- anual %>%
    filter(
        estacion ==
            "La Estanzuela"
    ) %>%
    drop_na(
        temp_media
    )
```

Podemos visualizar:

Tendencia visual

```
ggplot(
    tendencia_anual,
    aes(
        x = anio,
```

```
        y = temp_media
    )
) +
    geom_point() +
    geom_line() +
    geom_smooth(
        method = "lm",
        se = FALSE
    ) +
    labs(
        title =
            "Evolucion anual de la temperatura media",

        x =
            "Ano",

        y =
            "Temperatura media"
    ) +
    theme_minimal()
```

Idea clave

Una recta puede resumir una tendencia general, pero no implica que el comportamiento temporal sea realmente lineal.

16.23. Cuantificar una tendencia simple

Podemos ajustar:

Modelo de tendencia

```
modelo_tendencia <- lm(
    temp_media ~ anio,
    data = tendencia_anual
)

summary(
    modelo_tendencia
)
```

El coeficiente asociado a `anio` representa el cambio lineal estimado por unidad de tiempo.

Error frecuente

Interpretar automáticamente una tendencia observada como evidencia de una causa específica.

Buena práctica

Las tendencias deben interpretarse junto con el período observado, la calidad de los datos, posibles cambios de instrumentos y otras características del proceso de generación de los datos.

16.24. Comparar tendencias entre grupos

Podemos construir gráficos por estación:

Tendencias por estación

```
ggplot(
    anual,
    aes(
        x = anio,
        y = temp_media
    )
) +
    geom_point() +
    geom_line() +
    geom_smooth(
        method = "lm",
        se = FALSE
    ) +
    facet_wrap(
        ~ estacion
    ) +
    labs(
        title =
            "Tendencia de temperatura por estacion",

        x =
            "Ano",

        y =
            "Temperatura media"
    ) +
    theme_minimal()
```

Esto nos permite preguntar:

- ¿las estaciones presentan tendencias semejantes?;

- ¿alguna se comporta de forma claramente diferente?;
- ¿las diferencias aparecen durante todo el período o solamente en algunos años?;

16.25. Comparar contra un período de referencia

Otra estrategia consiste en definir un valor de referencia.

Por ejemplo, podemos comparar cada año con el primer año disponible.

Comparación con un valor inicial

```
comparacion_anual <- anual %>%
    group_by(
        estacion
    ) %>%

    arrange(
        anio,
        .by_group = TRUE
    ) %>%

    mutate(
        temp_referencia =
            first(
                temp_media
            ),

        diferencia_referencia =
            temp_media -
            temp_referencia
    ) %>%

    ungroup()
```

Idea clave

Expresar una serie como diferencia respecto a una referencia puede facilitar comparaciones entre grupos que poseen niveles absolutos diferentes.

16.26. Construir un índice base 100

En áreas como economía, producción o uso de recursos es frecuente representar una evolución mediante un índice.

Si el primer valor toma 100:

$$I_t = \frac{x_t}{x_0} \times 100.$$

En R:

Índice base 100

```
indice_anual <- anual %>%
    group_by(
        estacion
    ) %>%

    arrange(
        anio,
        .by_group = TRUE
    ) %>%

    mutate(
        valor_base =
            first(
                temp_media
            ),

        indice_temp =
            if_else(
                !is.na(valor_base) &
                valor_base != 0,

                temp_media /
                valor_base *
                100,

                NA_real_
            )
    ) %>%

    ungroup()
```

Buena práctica

Un índice elimina la escala absoluta y resalta el cambio relativo. Por eso puede ser útil para comparar evoluciones, pero no sustituye los valores originales.

16.27. La granularidad cambia la historia que vemos

Consideremos una misma variable observada:

- diariamente;
- mensualmente;
- anualmente.

Cada nivel puede mostrar patrones diferentes.

Los datos diarios preservan variaciones de corto plazo.

Los datos mensuales reducen ruido y facilitan observar patrones estacionales.

Los datos anuales permiten observar cambios de largo plazo, pero ocultan variaciones internas.

Idea clave

Agregar información siempre implica perder detalle. La pregunta es si ese detalle era relevante para el objetivo del análisis.

16.28. Una advertencia para modelos predictivos

Las operaciones temporales también pueden generar problemas si se utilizan incorrectamente en aprendizaje automático.

Supongamos que queremos predecir una variable en el tiempo t.

No podemos utilizar información del tiempo $t + 1$ como predictor.

Por ejemplo:

Una variable futura

```
datos <- datos %>%
    group_by(
        estacion
    ) %>%

    arrange(
        Fecha,
        .by_group = TRUE
    ) %>%

    mutate(
        temp_dia_siguiente =
            lead(
                temp_media
            )
    ) %>%

    ungroup()
```

Esta columna puede utilizarse como **variable objetivo futura**, pero no como predictor de la observación actual.

Error frecuente

Utilizar información futura durante el entrenamiento de un modelo produce fuga de información (*data leakage*) y puede generar resultados artificialmente buenos.

Idea clave

Cuando los datos tienen orden temporal debemos preguntarnos siempre qué información habría estado realmente disponible en el momento de realizar la predicción.

16.29. Un flujo de análisis temporal

Podemos resumir el procedimiento como:

Terminal

```
identificar variable temporal
        |
        v
verificar orden y periodos
        |
        v
definir granularidad
        |
        v
agrupar y resumir
        |
        v
verificar cobertura
        |
        v
comparar periodos
        |
        v
calcular diferencias / acumulados
        |
        v
suavizar si corresponde
        |
        v
visualizar patrones
        |
```

```
        v
comparar grupos
        |
        v
interpretar tendencias
```

Idea clave

La elección de una transformación temporal debe comenzar por la pregunta, no por la función que queremos utilizar.

16.30. Actividad para probar

Para probar

Cada grupo deberá utilizar una base vinculada a su proyecto y seleccionar al menos una variable cuya evolución temporal sea relevante para el problema estudiado.
El objetivo es construir una pequeña investigación sobre cómo cambia esa variable a lo largo del tiempo.

Para probar

El trabajo deberá incluir:

1. identificar claramente la variable temporal;
2. indicar cuál es la granularidad original de los datos;
3. revisar si existen períodos faltantes o incompletos;
4. construir al menos una nueva escala temporal mediante agregación;
5. justificar cómo se resume cada variable utilizada;
6. registrar cuántas observaciones participan en las agregaciones;
7. construir al menos una comparación entre períodos consecutivos utilizando `lag`;
8. calcular una diferencia absoluta o tasa de variación;
9. construir al menos un indicador acumulado o una media móvil, cuando tenga sentido para la base;
10. construir al menos dos visualizaciones temporales;
11. comparar la evolución entre grupos, regiones, estaciones, categorías u otras unidades relevantes, cuando corresponda;
12. discutir si aparece alguna tendencia o patrón temporal relevante.

Para probar

Además, respondan brevemente:

1. ¿qué cambia cuando observan los datos con diferente granularidad temporal?;
2. ¿existen períodos con menor cobertura de información?;
3. ¿las tendencias observadas son semejantes entre diferentes grupos?;

4. ¿aparecen períodos atípicos?;
5. ¿la media móvil ayuda a interpretar el comportamiento o elimina información importante?;
6. ¿qué hipótesis o preguntas nuevas surgen a partir del análisis temporal?;
7. ¿alguna de estas transformaciones podría ser útil posteriormente como entrada para un modelo predictivo?

Idea clave

El objetivo no es encontrar obligatoriamente una tendencia. Descubrir que una variable no presenta un comportamiento temporal claro también es un resultado del análisis.

16.31. Pensar más allá del ejemplo meteorológico

Las mismas herramientas pueden utilizarse en otros contextos.

Por ejemplo:

- evolución mensual de producción;
- cambios anuales en uso del suelo;
- crecimiento de una población;
- consumo energético;
- cantidad de transacciones;
- evolución de precios;
- comportamiento de indicadores productivos;
- utilización de recursos.

Podemos formular preguntas como:

> ¿La producción viene aumentando a lo largo del período?
>
> ¿Ese crecimiento ocurre de la misma forma en todas las regiones?
>
> ¿Los cambios de una variable coinciden temporalmente con cambios de otra?
>
> ¿El comportamiento observado corresponde a una tendencia general o solamente a algunos períodos particulares?

Idea clave

Una serie temporal no es interesante solamente porque contiene fechas. Es interesante cuando el orden temporal ayuda a responder una pregunta sobre el proceso estudiado.

16.32. Cierre de la sección

En esta sección incorporamos herramientas para estudiar la evolución temporal de los datos.

Aprendimos a cambiar la granularidad de una serie, construir agregaciones mensuales y anuales, verificar cobertura de información, comparar períodos mediante `lag` y `lead`, calcular variaciones, construir acumulados y utilizar medias móviles.

También exploramos patrones estacionales y tendencias simples, y discutimos cómo la escala temporal elegida puede modificar la interpretación de los datos.

Estas operaciones aparecen frecuentemente en proyectos reales de ciencia de datos y muchas veces deben repetirse para diferentes variables, estaciones, regiones, categorías o unidades de análisis.

En la próxima sección aprovecharemos justamente esa necesidad de repetición para avanzar hacia una programación más general: construiremos funciones reutilizables y utilizaremos herramientas de iteración funcional para automatizar análisis sobre múltiples grupos.

17. Automatización de análisis con funciones y purrr

17.1. Objetivo de la sección

A lo largo del curso construimos diferentes procedimientos de análisis:

- leer y preparar una base;
- filtrar observaciones;
- resumir información por grupos;
- construir gráficos;
- estudiar tendencias;
- ajustar regresiones;
- calcular métricas;
- aplicar clustering.

Muchas veces aparece ahora un nuevo problema:

> ¿Tenemos que repetir manualmente el mismo código para cada estación, región, categoría o variable?

La respuesta es no.

En esta sección aprenderemos a transformar procedimientos repetitivos en programas reutilizables.

Trabajaremos con:

- construcción de funciones;
- argumentos y valores de retorno;
- funciones que reciben tablas;
- iteración con `purrr`;
- `map`;
- `map_dbl`;
- `map_dfr`;
- `walk`;
- separación de una base por grupos;
- columnas que contienen objetos;
- ajuste automático de modelos por grupo;

- extracción automática de resultados;
- generación automática de gráficos;
- manejo básico de errores en procesos automatizados.

Idea clave

Automatizar no significa ejecutar más rápido un código repetido. Significa identificar qué parte del procedimiento cambia y qué parte permanece igual.

17.2. El problema de repetir código

Supongamos que queremos calcular un resumen meteorológico para una estación.

Podríamos escribir:

Resumen de una estación

```
datos %>%
    filter(
        estacion == "La Estanzuela"
    ) %>%
    summarise(
        registros = n(),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        precipitacion =
            sum(
                precipitacion,
                na.rm = TRUE
            )
    )
```

Si queremos hacer lo mismo para otra estación, podríamos copiar el código y cambiar el nombre:

Repetir el procedimiento

```
datos %>%
    filter(
        estacion == "Otra estacion"
    ) %>%
```

```
    summarise(
        registros = n(),

        temp_media =
            mean(
                temp_media,
                na.rm = TRUE
            ),

        precipitacion =
            sum(
                precipitacion,
                na.rm = TRUE
            )
    )
```

Y luego volver a copiarlo para una tercera estación.

Este procedimiento presenta varios problemas:

- repetimos código;
- aumentamos la posibilidad de cometer errores;
- cualquier modificación debe realizarse varias veces;
- el análisis resulta difícil de ampliar;
- el script se vuelve más largo y difícil de mantener.

Idea clave

Cuando encontramos bloques de código casi idénticos, probablemente existe una oportunidad para construir una función.

17.3. Construir una función

Podemos transformar el procedimiento anterior en:

Primera función

```
resumir_estacion <- function(
    datos,
    nombre_estacion
) {

    resultado <- datos %>%
        filter(
```

```
            estacion ==
                nombre_estacion
        ) %>%

        summarise(
            registros =
                n(),

            temp_media =
                mean(
                    temp_media,
                    na.rm = TRUE
                ),

            precipitacion =
                sum(
                    precipitacion,
                    na.rm = TRUE
                )
        )

    return(
        resultado
    )
}
```

Ahora podemos escribir:

Usar la función

```
resumir_estacion(
    datos,
    "La Estanzuela"
)
```

O:

Utilizar la misma función con otro valor

```
resumir_estacion(
    datos,
    "Otra estacion"
)
```

Cómo leer este código

La función tiene dos argumentos:

- `datos`: la tabla sobre la que trabajaremos;
- `nombre_estacion`: la estación que queremos analizar.

El procedimiento permanece igual.
Solamente cambia el valor del argumento.

17.4. ¿Qué debe convertirse en argumento?

Una buena pregunta al construir una función es:

> ¿Qué elemento podría cambiar la próxima vez que utilice este procedimiento?

Por ejemplo, podríamos permitir que cambie también el período.

Función con varios parámetros

```
resumir_estacion_periodo <- function(
    datos,
    nombre_estacion,
    fecha_inicio,
    fecha_fin
) {

    resultado <- datos %>%
        filter(
            estacion ==
                nombre_estacion,

            Fecha >=
                fecha_inicio,

            Fecha <=
                fecha_fin
        ) %>%

        summarise(
            registros =
                n(),

            temp_media =
                mean(
                    temp_media,
                    na.rm = TRUE
                ),

            precipitacion =
```

```
                    sum(
                        precipitacion,
                        na.rm = TRUE
                    )
            )

        return(
            resultado
        )
    }
```

Podemos utilizar:

Resumen parametrizado

```
resumir_estacion_periodo(
    datos = datos,
    nombre_estacion =
        "La Estanzuela",
    fecha_inicio =
        as.Date("2016-01-01"),
    fecha_fin =
        as.Date("2016-12-31")
)
```

Buena práctica

Una función debe recibir como argumentos los elementos que esperamos modificar, pero no es necesario convertir absolutamente todo en un parámetro.

17.5. Una función puede devolver diferentes tipos de objetos

Una función puede devolver:

- un número;
- un texto;
- un vector;
- una tabla;
- un gráfico;
- un modelo;
- una lista.

Por ejemplo:

Función que devuelve un número

```
calcular_mae <- function(
    observado,
    predicho
) {

    mean(
        abs(
            observado -
            predicho
        ),
        na.rm = TRUE
    )
}
```

Podemos usar:

Usar calcular_mae

```
calcular_mae(
    observado = evaluacion$temp_media,
    predicho = evaluacion$temp_predicha
)
```

Idea clave

Una función no tiene que imprimir un resultado. Debe devolver un objeto que pueda seguir utilizándose dentro del programa.

17.6. Acceder a un componente con $

Antes de continuar con funciones, conviene hacer explícita una notación de R que ya utilizamos varias veces durante el curso.

El operador `$` permite acceder a un componente de un objeto por su nombre.

Por ejemplo, si `evaluacion` es una tabla:

Acceder a una columna

```
evaluacion$temp_media
```

significa:

acceder a la columna `temp_media` de la tabla `evaluacion`.

De la misma forma:

Otra columna

```
evaluacion$temp_predicha
```

accede a la columna `temp_predicha`.

Por eso, en:

Utilizar columnas como argumentos

```
calcular_mae(
    evaluacion$temp_media,
    evaluacion$temp_predicha
)
```

el operador `$` no separa los argumentos.

Los argumentos están separados por la coma.

La llamada contiene dos expresiones:

1. `evaluacion$temp_media`;
2. `evaluacion$temp_predicha`.

Como no indicamos sus nombres, R los asocia con los parámetros de la función según su posición.

Si la función fue definida como:

Código R

```
calcular_mae <- function(
    observado,
    predicho
) {
    ...
}
```

entonces:

Código R

```
calcular_mae(
    evaluacion$temp_media,
    evaluacion$temp_predicha
)
```

es equivalente a escribir:

Código R

```
calcular_mae(
    observado = evaluacion$temp_media,
    predicho = evaluacion$temp_predicha
)
```

Idea clave

El operador `$` permite acceder a un componente de un objeto. La coma separa los argumentos de una función.

Buena práctica

Cuando una función tiene varios argumentos, escribir sus nombres puede hacer que la llamada sea más fácil de leer y reducir errores relacionados con el orden de los parámetros.

17.7. Una función que genera un gráfico

También podemos automatizar visualizaciones.

Función de visualización

```
graficar_temperatura <- function(
    datos,
    nombre_estacion
) {

    datos_estacion <- datos %>%
        filter(
            estacion ==
                nombre_estacion
        )

    grafico <- ggplot(
```

```
        datos_estacion,
        aes(
            x = Fecha,
            y = temp_media
        )
    ) +
        geom_line() +
        labs(
            title =
                "Temperatura media",

            subtitle =
                nombre_estacion,

            x =
                "Fecha",

            y =
                "Temperatura media"
        ) +
        theme_minimal()

    return(
        grafico
    )
}
```

Luego:

Generar un gráfico

```
graficar_temperatura(
    datos,
    "La Estanzuela"
)
```

17.8. De funciones a iteración

Construir una función resuelve una parte del problema.

Pero todavía podríamos terminar escribiendo:

Código R

```
resumir_estacion(datos, "Estacion A")
resumir_estacion(datos, "Estacion B")
resumir_estacion(datos, "Estacion C")
resumir_estacion(datos, "Estacion D")
```

Queremos ahora aplicar automáticamente la misma función a una colección de valores.

Para eso utilizaremos el paquete `purrr`.

Cargar purrr

```
library(purrr)
```

`purrr` también forma parte del ecosistema `tidyverse`.

17.9. La función map

Supongamos que queremos conocer los nombres de todas las estaciones.

Lista de estaciones

```
estaciones <- datos %>%
    distinct(
        estacion
    ) %>%
    pull(
        estacion
    )

print(
    estaciones
)
```

Podemos aplicar nuestra función a cada elemento:

Aplicar una función con map

```
resultados <- map(
    estaciones,
    ~ resumir_estacion(datos, .x)
)
```

Cómo leer este código

`map` recorre los elementos de `estaciones`.
En cada iteración:

Terminal

```
.x
```

representa el elemento actual.
Por lo tanto:

Terminal

```
resumir_estacion(datos, .x)
```

se ejecuta una vez para cada estación.

17.10. El resultado de map es una lista

Podemos inspeccionar:

Inspeccionar resultados

```
str(
    resultados
)
```

Una lista puede almacenar objetos de diferentes tipos y tamaños.

En este caso, cada elemento contiene una pequeña tabla.

Idea clave

`map` siempre devuelve una lista. Existen variantes cuando sabemos qué tipo de resultado queremos obtener.

17.11. Combinar resultados en una tabla

Como nuestra función devuelve una tabla, puede ser más conveniente utilizar:

Combinar resultados con map_dfr

```
resumen_estaciones <- map_dfr(
    estaciones,
    ~ resumir_estacion(datos, .x) %>%

        mutate(
            estacion = .x
        )
)

print(resumen_estaciones)
```

El resultado es una única tabla.

Cómo leer este código

`map_dfr` aplica una función a cada elemento y combina los resultados por filas.

Idea clave

La automatización resulta especialmente útil cuando el número de grupos puede cambiar. El código continúa siendo el mismo si tenemos 3, 7 o 100 estaciones.

17.12. Cuando esperamos un único número

Supongamos que queremos calcular la temperatura promedio de cada estación.

Creamos:

Función que devuelve un número

```
media_temperatura <- function(
    nombre_estacion
) {

    datos %>%
        filter(
            estacion ==
                nombre_estacion
        ) %>%

        summarise(
            valor =
                mean(
                    temp_media,
                    na.rm = TRUE
```

```
                    )
        ) %>%

        pull(
            valor
        )
}
```

Como sabemos que el resultado será numérico, podemos utilizar:

map_dbl

```
medias <- map_dbl(
    estaciones,
    media_temperatura
)

print(medias)
```

Cómo leer este código

`map_dbl` espera que cada ejecución produzca un único valor numérico.

Existen otras variantes:

Función	Resultado esperado
`map`	Lista.
`map_dbl`	Vector numérico.
`map_int`	Vector de enteros.
`map_chr`	Vector de texto.
`map_lgl`	Vector lógico.
`map_dfr`	Tablas combinadas por filas.

17.13. Separar una tabla por grupos

Otra estrategia consiste en dividir directamente la base.

Podemos escribir:

Separar por estación

```
datos_por_estacion <- datos %>%
    group_by(
        estacion
```

```
    ) %>%
    group_split()
```

El resultado es una lista de tablas.

Cantidad de grupos

```
length(
    datos_por_estacion
)
```

Podemos inspeccionar la primera:

Primer grupo

```
datos_por_estacion[[1]] %>%
    head()
```

Idea clave

Una lista de tablas permite tratar cada grupo como una pequeña base independiente.

17.14. Aplicar una función a cada tabla

Podemos construir una función que ya no necesite conocer el nombre de la estación:

Resumen de una tabla

```
resumir_grupo <- function(
    tabla
) {

    tabla %>%
        summarise(
            estacion =
                first(
                    estacion
                ),

            registros =
                n(),

            temp_media =
                mean(
```

```
                    temp_media,
                    na.rm = TRUE
                ),

            precipitacion =
                sum(
                    precipitacion,
                    na.rm = TRUE
                )
        )
}
```

Y aplicar:

Automatizar por grupo

```
resumen <- map_dfr(
    datos_por_estacion,
    resumir_grupo
)

print(resumen)
```

17.15. Una tabla que contiene tablas

Otra herramienta especialmente interesante es `group_nest`.

Podemos escribir:

Anidar por estación

```
datos_anidados <- datos %>%
    group_nest(
        estacion
    )

datos_anidados
```

Obtendremos conceptualmente algo como:

estacion	data
Estación A	<tibble>
Estación B	<tibble>
Estación C	<tibble>

La columna `data` es una **columna de listas**.

Cada celda contiene una tabla.

Idea clave

Una celda de un tibble no tiene que contener solamente un número o un texto. También puede contener objetos complejos, como tablas, modelos o gráficos.

17.16. Inspeccionar una tabla anidada

Podemos acceder a la primera tabla:

Contenido de una celda

```
datos_anidados$data[[1]] %>%
    head()
```

También podemos consultar su tamaño:

Cantidad de observaciones por tabla

```
datos_anidados <- datos_anidados %>%
    mutate(
        registros =
            map_int(
                data,
                nrow
            )
    )

datos_anidados %>%
    select(
        estacion,
        registros
    )
```

17.17. Ajustar una regresión para cada estación

Ahora podemos reutilizar la regresión estudiada anteriormente.

Queremos ajustar:

$$\text{temp_media} = \beta_0 + \beta_1 \text{hr_media} + \varepsilon$$

para cada estación.

Primero construimos una función:

Función para ajustar un modelo

```
ajustar_modelo <- function(
    tabla
) {

    datos_modelo <- tabla %>%
        drop_na(
            temp_media,
            hr_media
        )

    lm(
        temp_media ~ hr_media,
        data = datos_modelo
    )
}
```

Ahora:

Un modelo por estación

```
modelos_estacion <- datos %>%
    group_nest(
        estacion
    ) %>%

    mutate(
        modelo =
            map(
                data,
                ajustar_modelo
            )
    )

modelos_estacion
```

La tabla contiene ahora:

estacion	data	modelo
Estación A	<tibble>	<lm>
Estación B	<tibble>	<lm>
Estación C	<tibble>	<lm>

Idea clave

Acabamos de ajustar tantos modelos como estaciones existan sin escribir manualmente un modelo para cada una.

17.18. Transformar modelos en tablas

Los objetos generados por `lm` contienen mucha información.

Para trabajar cómodamente con ellos podemos utilizar el paquete `broom`.

Si fuera necesario instalarlo:

Instalar broom

```
install.packages("broom")
```

Luego:

Cargar broom

```
library(broom)
```

`broom` permite convertir resultados de modelos en tablas.

Por ejemplo:

Coeficientes de un modelo

```
tidy(
    modelos_estacion$modelo[[1]]
)
```

Y:

Indicadores generales del modelo

```
glance(
    modelos_estacion$modelo[[1]]
```

```
)
```

17.19. Extraer los coeficientes de todos los modelos

Podemos aplicar `tidy` a cada modelo:

Resultados de todos los modelos

```
coeficientes <- modelos_estacion %>%
    mutate(
        resultado =
            map(
                modelo,
                broom::tidy
            )
    ) %>%

    select(
        estacion,
        resultado
    ) %>%

    unnest(
        resultado
    )

print(
    coeficientes
)
```

Ahora tenemos una tabla convencional con columnas como:

- estación;
- término;
- estimación;
- error estándar;
- estadístico;
- valor asociado.

17.20. Comparar pendientes entre estaciones

Podemos seleccionar solamente el coeficiente asociado a humedad:

Pendiente por estación

```
pendientes <- coeficientes %>%
    filter(
        term ==
            "hr_media"
    ) %>%

    select(
        estacion,
        estimate
    ) %>%

    rename(
        pendiente =
            estimate
    )

print(
    pendientes
)
```

Podemos visualizar:

Comparar pendientes

```
ggplot(
    pendientes,
    aes(
        x = reorder(
            estacion,
            pendiente
        ),
        y = pendiente
    )
) +
    geom_col() +
    coord_flip() +
    labs(
        title =
            "Relacion estimada entre humedad y temperatura",

        x =
            "Estacion",

        y =
```

```
            "Pendiente estimada"
    ) +
    theme_minimal()
```

Idea clave

Automatizar el ajuste permite transformar una pregunta individual en una pregunta comparativa: ya no preguntamos solamente cuál es la relación en una estación, sino cómo cambia esa relación entre estaciones.

17.21. Comparar desempeño de los modelos

También podemos aplicar `glance`:

Resumen de todos los modelos

```
metricas_modelos <- modelos_estacion %>%
    mutate(
        metricas =
            map(
                modelo,
                broom::glance
            )
    ) %>%

    select(
        estacion,
        metricas
    ) %>%

    unnest(
        metricas
    )

metricas_modelos %>%
    select(
        estacion,
        r.squared,
        adj.r.squared,
        sigma,
        nobs
    )
```

Esto permite comparar los modelos de forma sistemática.

17.22. Automatizar una evaluación

También podemos escribir nuestras propias funciones.

Por ejemplo:

Calcular métricas de un modelo

```
evaluar_modelo <- function(
    modelo,
    tabla
) {

    predicciones <- predict(
        modelo,
        newdata = tabla
    )

    residuos <-
        tabla$temp_media -
        predicciones

    tibble(
        MAE =
            mean(
                abs(residuos),
                na.rm = TRUE
            ),

        RMSE =
            sqrt(
                mean(
                    residuos^2,
                    na.rm = TRUE
                )
            )
    )
}
```

Luego podríamos aplicar esta función a diferentes modelos y tablas.

17.23. Iterar sobre dos objetos simultáneamente

Hasta ahora `map` recorrió una sola colección.

Pero ahora necesitamos simultáneamente:

- un modelo;

- la tabla correspondiente.

Para eso podemos utilizar `map2`:

Evaluar cada modelo con su tabla

```
modelos_estacion <- modelos_estacion %>%
    mutate(
        evaluacion =
            map2(
                modelo,
                data,

                ~ evaluar_modelo(
                    .x,
                    .y
                )
            )
    )
```

Aquí:

- `.x` representa el modelo;
- `.y` representa la tabla correspondiente.

Luego:

Expandir métricas

```
metricas <- modelos_estacion %>%
    select(
        estacion,
        evaluacion
    ) %>%

    unnest(
        evaluacion
    )

print(
    metricas
)
```

17.24. Cuando queremos ejecutar una acción

Hasta ahora nos interesaba recuperar un resultado.

Sin embargo, algunas operaciones tienen como objetivo producir un efecto, por ejemplo:

- guardar un archivo;
- guardar un gráfico;
- escribir un mensaje.

Para estas situaciones podemos utilizar `walk`.

Supongamos:

Función para guardar un gráfico

```
guardar_grafico_estacion <- function(
    nombre_estacion
) {

    grafico <-
        graficar_temperatura(
            datos,
            nombre_estacion
        )

    nombre_archivo <-
        str_replace_all(
            nombre_estacion,
            " ",
            "_"
        )

    ggsave(
        filename =
            paste0(
                "outputs/figures/",
                nombre_archivo,
                ".png"
            ),

        plot =
            grafico,

        width =
            10,

        height =
            6,

        dpi =
```

```
            300
    )
}
```

Creamos la carpeta:

Crear carpeta de salida

```
dir.create(
    "outputs/figures",
    recursive = TRUE,
    showWarnings = FALSE
)
```

Y guardamos un gráfico para cada estación:

Guardar automáticamente todos los gráficos

```
walk(
    estaciones,
    guardar_grafico_estacion
)
```

Cómo leer este código

`walk` se utiliza cuando queremos ejecutar una operación por cada elemento, pero no necesitamos conservar una colección de valores de retorno.

17.25. Construir nombres de archivo automáticamente

Los nombres utilizados en los datos pueden contener espacios, símbolos o caracteres poco convenientes para archivos.

Podemos construir una función:

Preparar nombre para archivo

```
normalizar_nombre_archivo <- function(
    texto
) {

    texto %>%
        str_to_lower() %>%
        str_replace_all(
```

```
            "[^[:alnum:]]+",
            "_"
        ) %>%
        str_remove(
            "_$"
        )
}
```

Por ejemplo:

Ejemplo

```
normalizar_nombre_archivo(
    "La Estanzuela"
)
```

podría producir:

Terminal

```
la_estanzuela
```

Idea clave

La automatización también exige pensar en detalles que no aparecen cuando ejecutamos un procedimiento solamente una vez.

17.26. Automatizar un análisis para diferentes variables

La repetición no ocurre solamente entre grupos.

También puede ocurrir entre variables.

Supongamos que queremos calcular una media para:

- temperatura;
- humedad;
- radiación.

Podemos definir:

Lista de variables

```
variables <- c(
    "temp_media",
    "hr_media",
    "rad_solar"
)
```

Y construir una función que recibe el nombre de una columna:

Resumen de una variable

```
resumir_variable <- function(
    datos,
    variable
) {

    datos %>%
        summarise(
            variable =
                variable,

            media =
                mean(
                    .data[[variable]],
                    na.rm = TRUE
                ),

            minimo =
                min(
                    .data[[variable]],
                    na.rm = TRUE
                ),

            maximo =
                max(
                    .data[[variable]],
                    na.rm = TRUE
                )
        )
}
```

Luego:

Aplicar sobre varias variables

```
resumen_variables <- map_dfr(
    variables,

    ~ resumir_variable(
        datos,
        .x
    )
)

print(
    resumen_variables
)
```

Cómo leer este código

Cuando el nombre de una columna está almacenado como texto podemos utilizar:

Terminal

```
.data[[variable]]
```

para acceder a esa columna dentro de una operación de dplyr.

17.27. Automatizar por grupo y por variable

Podemos llevar la idea un paso más adelante.

Queremos calcular un resumen para cada combinación de:

- estación;
- variable.

Podemos crear todas las combinaciones:

Construir combinaciones

```
combinaciones <- crossing(
    estacion =
        estaciones,

    variable =
        variables
)

```

```
print(
    combinaciones
)
```

Luego construimos:

Función para grupo y variable

```
resumir_estacion_variable <- function(
    nombre_estacion,
    variable
) {

    datos %>%
        filter(
            estacion ==
                nombre_estacion
        ) %>%

        summarise(
            estacion =
                nombre_estacion,

            variable =
                variable,

            media =
                mean(
                    .data[[variable]],
                    na.rm = TRUE
                )
        )
}
```

Y utilizamos `pmap`:

Iterar sobre varios argumentos

```
resultados <- pmap_dfr(
    combinaciones,

    ~ resumir_estacion_variable(
        nombre_estacion = ..1,
        variable = ..2
    )
```

```
)

print(
    resultados
)
```

Idea clave

`map` recorre una colección, `map2` recorre dos y `pmap` permite recorrer varias colecciones en paralelo.

17.28. ¿Qué ocurre si una iteración falla?

Cuando ejecutamos una operación una sola vez, un error detiene el procedimiento.

En un proceso automatizado con decenas de grupos, puede resultar útil decidir qué hacer si uno de ellos presenta un problema.

Supongamos que una estación tiene muy pocas observaciones para ajustar un modelo.

Podemos mejorar nuestra función:

Validar antes de ajustar

```
ajustar_modelo_seguro <- function(
    tabla
) {

    datos_modelo <- tabla %>%
        drop_na(
            temp_media,
            hr_media
        )

    if (
        nrow(datos_modelo) < 10
    ) {

        return(
            NULL
        )
    }

    lm(
        temp_media ~ hr_media,
        data = datos_modelo
    )
```

```
}
```

Buena práctica

Una función robusta debe verificar las condiciones mínimas necesarias para realizar la operación.

17.29. Capturar errores con possibly

`purrr` también proporciona herramientas para manejar errores.

Por ejemplo:

Crear una versión tolerante a errores

```
ajustar_seguro <- possibly(
    ajustar_modelo,
    otherwise = NULL
)
```

Ahora:

Aplicar versión segura

```
modelos_estacion <- datos %>%
    group_nest(
        estacion
    ) %>%

    mutate(
        modelo =
            map(
                data,
                ajustar_seguro
            )
    )
```

Si una ejecución falla, el resultado correspondiente será `NULL` en lugar de detener todo el proceso.

Idea clave

Ignorar silenciosamente los errores no es una buena práctica. Una estrategia tolerante a errores debe acompañarse de un registro que permita identificar qué grupos fallaron.

17.30. Registrar el estado del procesamiento

Podemos agregar una columna que indique si el modelo fue generado:

Registrar éxito o falla

```
modelos_estacion <- modelos_estacion %>%
    mutate(
        modelo =
            map(
                data,
                ajustar_seguro
            ),

        estado =
            map_chr(
                modelo,

                ~ if (
                    is.null(.x)
                ) {
                    "error"
                } else {
                    "ok"
                }
            )
    )

modelos_estacion %>%
    count(
        estado
    )
```

Esto permite revisar los casos problemáticos:

Buscar grupos con problemas

```
modelos_estacion %>%
    filter(
        estado ==
            "error"
    ) %>%

    select(
        estacion,
        data
    )
```

17.31. Automatizar no significa dejar de revisar

Un código automatizado puede producir cien modelos en segundos.

Eso no significa que los cien resultados sean correctos o interpretables.

Debemos continuar revisando:

- cantidad de observaciones;
- valores faltantes;
- distribuciones;
- métricas;
- residuos;
- grupos con comportamientos inusuales;
- fallas de procesamiento.

Idea clave

La automatización permite ampliar un análisis, pero también puede ampliar nuestros errores.

17.32. Automatizar un análisis temporal por estación

Podemos combinar este capítulo con el anterior.

Construimos una función que genera un resumen mensual:

Resumen temporal reutilizable

```
construir_mensual <- function(
    tabla
) {

    tabla %>%
        mutate(
            mes =
                floor_date(
                    Fecha,
                    "month"
                )
        ) %>%

        group_by(
            mes
        ) %>%

```

```
        summarise(
            temp_media =
                mean(
                    temp_media,
                    na.rm = TRUE
                ),

            precipitacion =
                sum(
                    precipitacion,
                    na.rm = TRUE
                ),

            .groups =
                "drop"
        )
}
```

Y podemos producir una serie mensual para cada estación:

Automatizar series mensuales

```
series_mensuales <- datos %>%
    group_nest(
        estacion
    ) %>%

    mutate(
        mensual =
            map(
                data,
                construir_mensual
            )
    )

series_mensuales
```

Podemos expandirlas nuevamente:

Reunir las series

```
mensual_todas <- series_mensuales %>%
    select(
        estacion,
        mensual
```

```
    ) %>%

    unnest(
        mensual
    )

glimpse(
    mensual_todas
)
```

17.33. Del análisis manual al análisis automatizado

Podemos representar la evolución de nuestro código:

Terminal

```
resolver un caso
        |
        v
identificar partes repetidas
        |
        v
construir una funcion
        |
        v
definir argumentos
        |
        v
probar la funcion con un caso
        |
        v
aplicar sobre varios casos
        |
        v
combinar resultados
        |
        v
validar resultados
        |
        v
registrar errores
        |
        v
exportar automaticamente
```

Idea clave

Primero debemos conseguir que el procedimiento funcione correctamente para un caso. Recién después conviene automatizarlo.

17.34. ¿Cuándo conviene construir una función?

Algunas señales útiles son:

- copiamos y pegamos el mismo código;
- solamente cambia un nombre, fecha o variable;
- queremos repetir el análisis para muchos grupos;
- necesitamos aplicar exactamente las mismas reglas varias veces;
- queremos probar el procedimiento en otras bases;
- el código puede describirse como una operación con entradas y una salida.

No es necesario convertir cada línea del programa en una función.

Buena práctica

Una función debe representar una tarea coherente y tener un propósito fácil de explicar.

17.35. Elegir nombres informativos

Comparemos:

Terminal

```
f1()
hacer()
funcion_nueva()
```

con:

Terminal

```
calcular_metricas()
construir_resumen_mensual()
ajustar_modelo()
graficar_estacion()
exportar_resultados()
```

Buena práctica

Un buen nombre permite comprender qué hace una función sin necesidad de leer inmediatamente todo su código.

17.36. Actividad para probar

Para probar

Cada grupo deberá retomar alguna parte de los análisis desarrollados durante el curso sobre la base vinculada a su proyecto.
El objetivo será identificar un procedimiento que actualmente se repite para diferentes grupos, variables, regiones, períodos, estaciones, categorías u otras unidades y transformarlo en un flujo automatizado.

Para probar

El trabajo deberá incluir, como mínimo:

1. identificar claramente qué procedimiento se desea automatizar;
2. mostrar brevemente cómo sería realizado manualmente para un único caso;
3. construir al menos dos funciones reutilizables;
4. definir claramente los argumentos de cada función;
5. hacer que las funciones devuelvan objetos que puedan continuar utilizándose;
6. utilizar al menos una función de la familia `map`;
7. automatizar el procedimiento para al menos tres grupos, variables o unidades diferentes;
8. reunir los resultados en una tabla;
9. realizar alguna comparación entre los resultados obtenidos automáticamente;
10. incluir al menos una validación que permita detectar situaciones problemáticas.

Para probar

Además, seleccionen al menos una de las siguientes extensiones:

1. ajustar automáticamente un modelo de regresión para varios grupos;
2. construir automáticamente un resumen temporal para varios grupos;
3. generar y guardar automáticamente varios gráficos;
4. calcular automáticamente indicadores para varias variables;
5. ejecutar una combinación de grupo y variable utilizando `map2` o `pmap`.

Para probar

En comentarios dentro del código, respondan:

1. ¿qué parte del análisis se repetía originalmente?;
2. ¿qué elementos se convirtieron en argumentos?;
3. ¿qué devuelve cada función?;
4. ¿por qué eligieron `map`, `map_dbl`, `map_dfr`, `map2`, `pmap` o `walk`?;

5. ¿qué problema habría tenido realizar todo manualmente?;
6. ¿qué validaciones incorporaron?;
7. ¿qué podría ocurrir si uno de los grupos presenta datos inesperados?;
8. ¿cómo verificarían que la automatización no produjo resultados incorrectos?

Idea clave

El objetivo no es escribir el código más corto posible. El objetivo es construir un procedimiento que pueda reutilizarse, ampliarse y verificarse.

17.37. Un ejemplo de flujo automatizado

Al final de esta sección ya podemos imaginar un flujo como:

Terminal

```
leer base
    |
    v
preparar datos
    |
    v
identificar grupos
    |
    v
dividir base
    |
    v
aplicar funcion de analisis
    |
    +----> grupo 1
    |
    +----> grupo 2
    |
    +----> grupo 3
    |
    +----> ...
    |
    v
reunir resultados
    |
    v
comparar
    |
    v
generar graficos
    |
```

```
      v
exportar tablas y figuras
```

Este tipo de organización será particularmente importante para el proyecto final.

17.38. Cierre de la sección

En esta sección pasamos de ejecutar análisis individuales a construir procedimientos reutilizables y automatizados.

Utilizamos funciones para encapsular tareas, identificamos qué elementos deben convertirse en argumentos y trabajamos con la familia de funciones `map` para repetir operaciones sin copiar código.

También aprendimos a dividir una base por grupos, almacenar tablas y modelos dentro de columnas, ajustar automáticamente regresiones para diferentes grupos, extraer resultados mediante `broom`, generar archivos y gráficos en forma automática y considerar situaciones donde algunas iteraciones pueden fallar.

Estas herramientas permiten transformar una colección de scripts independientes en un verdadero flujo de procesamiento.

El siguiente paso será integrar todos estos elementos dentro de un proyecto reproducible. Organizaremos datos, scripts, funciones, tablas, figuras y resultados, y utilizaremos Quarto para generar automáticamente un documento que combine código, análisis y comunicación de resultados.

18. Proyectos reproducibles y comunicación de resultados con Quarto

18.1. Objetivo de la sección

A lo largo del curso trabajamos con diferentes partes de un flujo de ciencia de datos:

- lectura de archivos;
- diagnóstico de bases;
- limpieza y transformación;
- visualización;
- bases de datos relacionales y SQL;
- regresión;
- clustering;
- análisis temporal;
- construcción de funciones;
- automatización de análisis.

Hasta ahora muchas de estas tareas fueron desarrolladas en ejercicios o scripts separados.

En esta sección vamos a reunirlas dentro de una idea más general:

construir un proyecto reproducible de ciencia de datos.

Un proyecto reproducible debería permitir que otra persona, utilizando los mismos datos y el mismo código, pueda reconstruir los principales resultados del análisis.

Trabajaremos con:

- organización del proyecto;
- separación entre datos originales y procesados;
- scripts y funciones reutilizables;
- caminos relativos;
- documentación;
- generación automática de resultados;
- informes con Quarto;
- integración de texto, código, tablas y gráficos;
- registro de decisiones y limitaciones.

Idea clave

Un análisis reproducible no es solamente aquel cuyo código funciona en nuestra computadora. Debe existir una estructura clara que permita reconstruir el proceso desde los datos originales hasta los resultados finales.

18.2. ¿Qué significa reproducible?

Supongamos que recibimos un informe con un gráfico interesante.

Podemos preguntarnos:

- ¿de dónde salieron los datos?;
- ¿qué transformaciones se aplicaron?;
- ¿qué observaciones fueron eliminadas?;
- ¿cómo se construyeron las variables derivadas?;
- ¿qué modelo fue utilizado?;
- ¿con qué parámetros?;
- ¿cómo se generó el gráfico?;

Si estas decisiones no están registradas, reconstruir el resultado puede ser difícil o imposible.

Idea clave

Reproducibilidad significa poder reconstruir un resultado a partir de sus datos, código y decisiones documentadas.

18.3. Un flujo difícil de reproducir

Un proyecto puede terminar organizado de esta forma:

Terminal

```
proyecto/
|
|-- datos_finales.csv
|-- datos_finales2.csv
|-- datos_finales_ahora_si.csv
|-- script.R
|-- script_nuevo.R
|-- prueba.R
|-- grafico1.png
|-- grafico_final.png
|-- grafico_final2.png
```

```
|-- resultados.xlsx
```

Puede funcionar para quien creó los archivos, pero resulta difícil responder:

- cuál es la base original;
- cuál es la última versión;
- qué script genera cada resultado;
- qué archivos pueden eliminarse;
- qué debe ejecutarse primero.

Error frecuente

Utilizar los nombres de los archivos como sistema de control de versiones: `final`, `final2`, `definitivo`, `ahora_si`, etc.

18.4. Una estructura de proyecto

Una posible organización es:

Estructura de un proyecto reproducible

```
proyecto/
|
|-- data/
|   |-- raw/
|   |-- processed/
|
|-- R/
|   |-- funciones.R
|
|-- scripts/
|   |-- 01_preparar_datos.R
|   |-- 02_analizar.R
|   |-- 03_generar_resultados.R
|
|-- sql/
|   |-- consultas.sql
|
|-- outputs/
|   |-- figures/
|   |-- tables/
|
|-- report/
|   |-- informe.qmd
```

```
|
|-- README.md
```

Idea clave

La estructura debe ayudar a comprender el flujo del proyecto sin necesidad de abrir todos los archivos.

18.5. Datos originales y datos procesados

Una regla importante es separar:

Terminal

```
data/raw/
```

de:

Terminal

```
data/processed/
```

Los archivos dentro de `raw` representan los datos originales recibidos o descargados.

No deberían ser modificados manualmente por los scripts de análisis.

Las transformaciones generan nuevos archivos dentro de `processed`.

Buena práctica

Si algo sale mal durante el procesamiento, siempre debemos poder regresar a los datos originales.

18.6. Evitar caminos absolutos

Un código como:

Código R

```r
datos <- read_csv(
    "C:/Users/Marcelo/Desktop/proyecto/datos.csv"
)
```

puede funcionar en una computadora, pero probablemente no funcionará en otra.

Preferimos caminos relativos:

Utilizar caminos relativos

```
datos <- read_csv(
    "data/raw/datos.csv"
)
```

Idea clave

Los caminos relativos describen dónde está un archivo dentro del proyecto, no dónde está almacenado el proyecto dentro de una computadora específica.

18.7. Construir caminos con here

Otra posibilidad es utilizar el paquete `here`.

Si fuera necesario:

Código R

```
install.packages("here")
```

Luego:

Construir un camino

```
library(here)

ruta <- here(
    "data",
    "raw",
    "datos.csv"
)

datos <- read_csv(
    ruta
)
```

Esto facilita la construcción de caminos dentro del proyecto.

18.8. Separar las etapas del proceso

Un único script de miles de líneas puede ser difícil de mantener.

Podemos separar el flujo en etapas:

Terminal

```
01_preparar_datos.R
        |
        v
datos procesados
        |
        v
02_analizar.R
        |
        v
resultados
        |
        v
03_generar_resultados.R
```

Por ejemplo:

Terminal

```
01_preparar_datos.R
```

podría:

- leer los datos originales;
- limpiar nombres;
- convertir fechas;
- construir variables;
- validar registros;
- exportar la base procesada.

Luego:

Terminal

```
02_analizar.R
```

podría:

- leer la base procesada;
- construir resúmenes;

- realizar análisis temporal;
- ajustar modelos;
- generar resultados.

18.9. Separar funciones del flujo principal

Las funciones reutilizables pueden guardarse en:

Terminal

```
R/funciones.R
```

Por ejemplo:

Archivo de funciones

```
calcular_mae <- function(
    observado,
    predicho
) {

    mean(
        abs(
            observado -
            predicho
        ),
        na.rm = TRUE
    )
}


calcular_rmse <- function(
    observado,
    predicho
) {

    sqrt(
        mean(
            (
                observado -
                predicho
            )^2,
            na.rm = TRUE
        )
    )
}
```

Luego podemos cargar esas funciones desde otro script:

Cargar funciones

```
source(
    "R/funciones.R"
)
```

Idea clave

El script principal describe el flujo del análisis. Las funciones encapsulan tareas que pueden reutilizarse.

18.10. El orden debe quedar claro

Los nombres:

Terminal

```
01_preparar_datos.R
02_analizar.R
03_generar_resultados.R
```

permiten comprender inmediatamente el orden esperado.

No es la única forma posible de organizar un proyecto, pero es una estrategia simple y efectiva.

Buena práctica

Una persona que abre el proyecto por primera vez debería poder descubrir rápidamente por dónde comenzar.

18.11. Los resultados también deben ser reproducibles

Las figuras y tablas importantes deberían ser generadas mediante código.

Por ejemplo:

Guardar una tabla

```
write_csv(
    resumen,
```

```
    "outputs/tables/resumen.csv"
)
```

O:

Guardar un gráfico

```
ggsave(
    filename =
        "outputs/figures/tendencia.png",

    plot =
        grafico_tendencia,

    width =
        10,

    height =
        6,

    dpi =
        300
)
```

Idea clave

Si cambia la base de datos, debemos poder regenerar automáticamente las tablas y figuras.

18.12. Documentar el proyecto con README

El archivo:

Terminal

```
README.md
```

debe explicar brevemente el proyecto.

Puede contener:

- objetivo;
- origen de los datos;
- estructura de directorios;

- herramientas necesarias;
- orden de ejecución;
- principales resultados;
- limitaciones conocidas.

Una estructura simple podría ser:

Terminal

```
Nombre del proyecto

Objetivo

Datos

Estructura del proyecto

Requisitos

Ejecucion

Resultados principales

Limitaciones
```

Buena práctica

El README no debe explicar cada línea del código. Debe permitir comprender qué es el proyecto y cómo utilizarlo.

18.13. Primer contacto con Quarto

Antes de utilizar Quarto para construir un informe completo, vamos a realizar un ejemplo mínimo.

La idea será:

1. instalar y verificar las herramientas necesarias;
2. crear un pequeño documento `.qmd`;
3. definir un valor;
4. definir una función;
5. aplicar la función;
6. generar un informe HTML con el resultado.

Idea clave

Quarto no requiere RStudio. En este curso continuaremos utilizando R desde RGui y utilizaremos el paquete `quarto` como interfaz para generar los documentos.

18.13.1. Instalar Quarto

Quarto es una aplicación independiente de R.

Por lo tanto, primero debemos instalar Quarto en el sistema operativo. Podemos obtener el instalador correspondiente desde https://quarto.org/docs/get-started/.

Después de instalarlo, podemos comprobar desde R si Quarto fue encontrado.

Primero instalamos el paquete de R que utilizaremos como interfaz:

Instalar el paquete quarto

```
install.packages("quarto")
```

Para trabajar con documentos que ejecutan código R también instalamos:

Instalar soporte para documentos con R

```
install.packages("rmarkdown")
```

El paquete `rmarkdown` instala también `knitr`, que será utilizado para ejecutar los bloques de código R incluidos en el documento.

Estas instalaciones se realizan solamente una vez.

Luego podemos cargar el paquete:

Cargar quarto

```
library(quarto)
```

Y verificar si Quarto está disponible:

Verificar la instalación

```
quarto_available()
```

Si la instalación fue reconocida correctamente, el resultado será:

Terminal

```
TRUE
```

También podemos consultar la versión:

Consultar la versión

```
quarto_version()
```

Idea clave

El paquete `quarto` de R no es el programa Quarto. Es una interfaz que permite utilizar Quarto desde el propio entorno de R.

18.13.2. Crear nuestro primer documento

Crearemos un archivo de texto llamado:

Terminal

```
primer_informe.qmd
```

El archivo tendrá inicialmente la siguiente estructura:

Terminal

```
---
title: "Mi primer informe con Quarto"
format: html
---

# Un calculo sencillo

En este documento vamos a definir un valor,
construir una funcion y mostrar su resultado.
```

La parte ubicada entre las dos líneas de tres guiones corresponde al encabezado del documento, donde indicamos, entre otras cosas, el título y el formato de salida.

En este caso:

Terminal

```
format: html
```

indica que queremos generar una página HTML.

18.13.3. Agregar código R

Dentro de un documento Quarto podemos incluir bloques de código R.

Un bloque de R comienza con:

Terminal

```
```{r}
```

y termina con:

Terminal

```
```
```

Dentro de ese bloque escribimos código R normalmente.

Para nuestro primer ejemplo utilizaremos:

Código del primer informe

```r
valor <- 5

cuadrado <- function(
    x
) {
    x^2
}

resultado <- cuadrado(
    valor
)

resultado
```

Por lo tanto, dentro del archivo `primer_informe.qmd`, este código deberá estar ubicado entre los delimitadores del bloque R.

18.13.4. El archivo completo

Nuestro primer archivo `primer_informe.qmd` tendrá entonces la siguiente estructura:

Terminal

````
---
title: "Mi primer informe con Quarto"
format: html
---

# Un calculo sencillo

Vamos a definir un valor y aplicar una funcion.

```{r}
valor <- 5

cuadrado <- function(x) {
    x^2
}

resultado <- cuadrado(valor)

resultado
```

El resultado anterior fue calculado automaticamente
durante la generacion del informe.
````

Cómo leer este código

El archivo combina dos tipos de contenido:

- texto escrito en Markdown;
- código R contenido dentro de bloques ejecutables.

Cuando Quarto genera el documento, ejecuta el código y coloca su salida en el informe.

18.13.5. Generar el informe desde RGui

Guardamos el archivo `primer_informe.qmd` dentro de nuestra carpeta de trabajo.

Desde RGui podemos comprobar primero cuál es el directorio actual:

Cambiar y consultar el directorio de trabajo

```
setwd("C:/PACD")
getwd()
```

Si el archivo se encuentra en ese directorio, podemos generar el informe utilizando:

Generar el informe

```
quarto::quarto_render("primer_informe.qmd")
```

Quarto ejecutará el documento y generará:

Terminal

```
primer_informe.html
```

en la misma carpeta.

Podemos abrir ese archivo con un navegador web.

18.13.6. ¿Qué ocurrió?

Durante la generación del documento, Quarto ejecutó:

Código R

```
valor <- 5
```

Luego definió:

Código R

```
cuadrado <- function(
    x
) {
    x^2
}
```

y finalmente calculó:

Código R

```
resultado <- cuadrado(
    valor
)
```

Como:

$$5^2 = 25,$$

el informe generado mostrará el resultado:

Terminal

```
25
```

Idea clave

El valor mostrado en el informe no fue escrito manualmente. Fue producido por el código durante la generación del documento.

18.13.7. Modificar el dato y volver a generar

Ahora podemos modificar solamente:

Código R

```
valor <- 8
```

y ejecutar nuevamente:

Código R

```
quarto::quarto_render(
    "primer_informe.qmd"
)
```

El informe será reconstruido utilizando el nuevo valor.

Como:

$$8^2 = 64,$$

el nuevo resultado será:

Terminal

```
64
```

No necesitamos modificar manualmente el resultado dentro del informe.

Idea clave

Esta es la idea fundamental de un informe reproducible: cuando cambian los datos o el código, el documento puede reconstruirse automáticamente a partir de ellos.

El ejemplo anterior fue deliberadamente simple. En un proyecto real, en lugar de calcular el cuadrado de un número, podemos utilizar los mismos mecanismos para leer una base procesada, generar tablas, construir gráficos y mostrar resultados de los análisis realizados durante el curso.

18.14. Comunicar resultados con Quarto

Hasta ahora generamos:

- código;
- tablas;
- gráficos;
- resultados de modelos.

Pero normalmente queremos presentar estos elementos acompañados por una explicación.

Quarto permite combinar en un mismo documento:

- texto;
- código;
- resultados;
- tablas;
- gráficos;
- expresiones matemáticas.

Idea clave

Quarto no depende de RStudio. Un documento `.qmd` puede escribirse con diferentes editores y generarse desde la línea de comandos o desde R. RStudio ofrece una integración cómoda con Quarto, pero no es necesario para construir informes reproducibles.

Un documento Quarto utiliza normalmente la extensión:

Terminal

```
.qmd
```

18.15. Estructura básica de un documento Quarto

Un archivo puede comenzar con:

Encabezado de un archivo Quarto

```
---
title: "Analisis de datos"
author: "Grupo"
format: html
---
```

Luego escribimos texto utilizando Markdown.

Por ejemplo:

Terminal

```
Introduccion

En este informe analizamos...

Datos

La base contiene...

Resultados

Los principales resultados fueron...
```

18.16. Incluir código R

Podemos agregar código dentro del documento:

Bloque de código R

```
```{r}
library(tidyverse)
```
```

````
datos <- read_csv(
    "../data/processed/datos.csv"
)

glimpse(datos)
```
````

Cuando Quarto genera el documento, ejecuta el código e incorpora el resultado.

Idea clave

El informe deja de ser un documento separado del análisis. El propio código genera los resultados que aparecen dentro del informe.

18.17. Controlar qué aparece en el informe

No siempre queremos mostrar cada línea de código.

Podemos utilizar opciones como, por ejemplo:

Terminal

````
```{r}
#| echo: false
#| warning: false

ggplot(
    datos,
    aes(
        x = fecha,
        y = valor
    )
) +
    geom_line()
```
````

El código se ejecuta, pero solamente aparece el resultado.

Buena práctica

Un informe reproducible no necesita mostrar absolutamente todo el código. El código completo puede permanecer en los scripts del proyecto.
````

18.18. Generar gráficos directamente dentro del informe

Podemos utilizar:

Terminal

````
```{r}
#| label: fig-tendencia
#| fig-cap: "Evolucion de la variable durante el periodo."

ggplot(
    datos,
    aes(
        x = fecha,
        y = valor
    )
) +
    geom_line() +
    theme_minimal()
```
````

Así, el gráfico es generado durante la construcción del documento.

Idea clave

El informe y el análisis permanecen sincronizados porque las figuras se generan a partir del mismo código.

18.19. Incluir tablas

También podemos mostrar tablas calculadas durante el análisis.

Por ejemplo:

Terminal

````
```{r}
resumen <- datos %>%
    group_by(grupo) %>%
    summarise(
        media = mean(
            valor,
            na.rm = TRUE
        ),
        .groups = "drop"
    )

knitr::kable(
````
````

````
    resumen
)
```
````

18.20. Resultados dentro del texto

También podemos incorporar resultados directamente dentro de una frase.

Supongamos:

Código R

```r
cantidad <- nrow(datos)
```

En Quarto podemos utilizar un resultado calculado dentro del texto.

Esto evita escribir manualmente números que podrían cambiar cuando cambia la base.

Idea clave

Todo resultado que pueda cambiar cuando cambian los datos debería, siempre que sea razonable, generarse a partir del código.

18.21. Una posible estructura del informe

Para un proyecto de ciencia de datos podemos utilizar:

Terminal

```
Introduccion

Pregunta de analisis

Datos

Preparacion y calidad de los datos

Analisis exploratorio

Analisis aplicado

Resultados

Limitaciones
```
```

```
Conclusiones
```

No todas las secciones necesitan tener la misma extensión.

Buena práctica

El informe debe contar una historia sobre el problema y los resultados. No debe convertirse en una cronología de todas las instrucciones ejecutadas.

18.22. La prueba más importante

Antes de considerar terminado el proyecto podemos realizar una prueba:

> ¿Podemos reconstruir los resultados partiendo solamente de la base original y del código?

Una buena prueba consiste en:

1. eliminar temporalmente los archivos generados;
2. conservar los datos originales;
3. ejecutar el flujo desde el comienzo;
4. verificar que se reconstruyan los datos procesados;
5. verificar las tablas;
6. verificar los gráficos;
7. generar nuevamente el informe.

Idea clave

"Funciona en mi computadora" no es una prueba suficiente de reproducibilidad.

18.23. Incorporar validaciones

Un pipeline reproducible también debería detectar situaciones inesperadas.

Por ejemplo:

Verificar columnas necesarias

```
columnas_necesarias <- c(
    "Fecha",
    "estacion",
    "temp_media"
)
```

```r

faltantes <- setdiff(
    columnas_necesarias,
    names(datos)
)

if (
    length(faltantes) > 0
) {

    stop(
        paste(
            "Faltan columnas:",
            paste(
                faltantes,
                collapse = ", "
            )
        )
    )
}
```

Podemos también verificar:

- cantidad mínima de observaciones;
- rangos esperados;
- identificadores duplicados;
- fechas válidas;
- resultados de un `JOIN`;
- existencia de archivos.

Idea clave

Un pipeline robusto no solamente produce resultados cuando todo está bien. También debe avisarnos cuando encuentra algo inesperado.

18.24. Registrar las dependencias

El proyecto puede utilizar paquetes como:

- `tidyverse`;
- `lubridate`;
- `slider`;

- `purrr`;
- `broom`;
- `DBI`;
- `RMariaDB`;
- `here`.

Estas dependencias deberían quedar registradas en el README o en una sección específica del proyecto.

También podemos registrar información de la sesión:

Información del entorno

```
sessionInfo()
```

Esto permite conocer las versiones de R y de los paquetes utilizados.

18.25. Reproducibilidad y aleatoriedad

Algunos procedimientos contienen componentes aleatorios.

Por ejemplo:

Código R

```
kmeans(
    datos,
    centers = 3
)
```

puede depender de la inicialización.

Por eso utilizamos:

Fijar una semilla

```
set.seed(123)
```

antes de realizar procedimientos aleatorios.

Buena práctica

Si una parte del análisis utiliza aleatoriedad, debemos controlar esa aleatoriedad cuando queremos reproducir exactamente los resultados.

18.26. El proyecto como pipeline

Podemos representar un proyecto completo como:

Terminal

```
datos originales
      |
      v
diagnostico
      |
      v
limpieza y validacion
      |
      v
datos procesados
      |
      v
transformacion
      |
      +----------> SQL / base relacional
      |
      v
analisis exploratorio
      |
      v
analisis temporal
      |
      v
regresion / clustering / otros metodos
      |
      v
automatizacion
      |
      v
tablas y graficos
      |
      v
informe reproducible
```

Idea clave

El producto de un proyecto de ciencia de datos no es solamente el resultado final. También es el proceso que permite producirlo nuevamente.

18.27. Actividad final de la unidad curricular

Para probar

La última actividad de Programación Avanzada para Ciencia de Datos consiste en construir un pequeño proyecto reproducible utilizando una base vinculada al proyecto de aprendizaje automático del grupo.
No se busca repetir ni sustituir el proyecto de aprendizaje automático.
El objeto de evaluación en esta unidad curricular será el **pipeline computacional de datos**: cómo se organiza, transforma, automatiza, analiza y documenta la información.

Para probar

Cada grupo deberá partir de una base original y construir un flujo que permita llegar de forma reproducible hasta un conjunto de resultados.
El proyecto deberá poder ejecutarse nuevamente sin modificar manualmente archivos intermedios.

18.28. Requisitos mínimos de la actividad final

Para probar

El proyecto deberá incluir, como mínimo:

1. una estructura organizada de directorios;
2. conservación de los datos originales en una carpeta separada;
3. al menos un script encargado de preparar los datos;
4. generación automática de una base procesada;
5. al menos dos validaciones sobre estructura o calidad de los datos;
6. utilización de funciones propias para encapsular tareas del procesamiento o análisis;
7. al menos una automatización mediante funciones de la familia `map` u otra estrategia equivalente;
8. al menos una transformación temporal, agrupación o construcción de indicadores relevante para el problema;
9. al menos dos visualizaciones generadas mediante código;
10. al menos una consulta SQL relevante o una interacción con la estructura relacional trabajada durante el curso;
11. al menos una técnica de análisis aplicado estudiada durante el semestre, por ejemplo:
    - regresión;

- clustering;
- análisis temporal de tendencias;

12. generación automática de tablas o archivos de resultados;
13. un archivo `README.md`;
14. un informe reproducible construido con Quarto.

Idea clave

La actividad final no consiste en utilizar obligatoriamente todas las funciones o técnicas estudiadas durante el semestre.
Las herramientas deben seleccionarse porque aportan algo al problema analizado.

Por ejemplo, no es necesario aplicar clustering si no existe una pregunta razonable de agrupamiento.

Tampoco es necesario construir un modelo complejo solamente para demostrar que sabemos utilizar una función.

18.29. Informe reproducible

Para probar

El informe Quarto deberá incluir, como mínimo:

1. una breve descripción del problema;
2. origen y características generales de los datos;
3. descripción de las principales decisiones de preparación;
4. observaciones sobre calidad y limitaciones de la base;
5. al menos una pregunta de análisis claramente formulada;
6. resultados acompañados por tablas o gráficos;
7. interpretación de los resultados;
8. limitaciones del análisis;
9. conclusiones.

Buena práctica

El informe no debe ser una impresión completa de la consola ni una copia del código.
Debe seleccionar y comunicar los elementos necesarios para comprender el análisis.

18.30. README del proyecto

Para probar

El archivo `README.md` deberá permitir que otra persona comprenda:

1. cuál es el objetivo del proyecto;
2. qué datos utiliza;
3. cómo está organizada la estructura de carpetas;

4. qué paquetes o herramientas son necesarios;
5. cuál es el orden de ejecución;
6. dónde se encuentran los resultados;
7. cómo generar el informe final.

18.31. Calidad del código

Para probar

Durante la evaluación también se considerará:

- claridad de los nombres utilizados;
- organización del código;
- uso razonable de funciones;
- ausencia de bloques innecesariamente duplicados;
- utilización de caminos relativos;
- comentarios útiles;
- separación entre datos originales y procesados;
- posibilidad de ejecutar nuevamente el flujo;
- manejo razonable de situaciones inesperadas.

18.32. Resultados y descubrimientos

Para probar

No se evaluará el proyecto según si encuentra una relación fuerte, un modelo con bajo error o clusters perfectamente separados.
Se valorará que el grupo sea capaz de:

- formular preguntas coherentes;
- elegir herramientas apropiadas;
- interpretar los resultados;
- identificar resultados inesperados;
- reconocer limitaciones;
- explicar qué descubrió durante el proceso.

Idea clave

Un resultado negativo, débil o inesperado puede ser un buen resultado científico si fue obtenido mediante un procedimiento correcto y es interpretado adecuadamente.

18.33. Elementos a entregar

Para probar

Cada grupo deberá entregar una carpeta comprimida con la estructura completa del proyecto, incluyendo:

- scripts;
- funciones;
- consultas SQL utilizadas;
- datos procesados necesarios;
- README;
- archivo fuente de Quarto;
- informe generado;
- tablas y figuras generadas por el flujo.

Cuando la base original sea demasiado grande para incluirla en la entrega, deberá indicarse claramente en el README cómo obtenerla o dónde debe ubicarse para reproducir el proyecto.

18.34. Presentación final

Para probar

Cada grupo realizará una presentación breve del proyecto.
La presentación deberá concentrarse en:

1. la pregunta o problema seleccionado;
2. la estructura del pipeline;
3. las principales decisiones de preparación de datos;
4. alguna función o automatización que consideren particularmente relevante;
5. los principales resultados obtenidos;
6. al menos una visualización;
7. alguna dificultad encontrada durante el desarrollo;
8. una limitación del análisis;
9. una mejora que implementarían si continuaran trabajando en el proyecto.

Buena práctica

Durante la presentación no es necesario recorrer línea por línea todo el código. Es preferible seleccionar algunos fragmentos que permitan explicar las decisiones de programación más importantes.

18.35. Prueba final antes de entregar

Para probar

Antes de realizar la entrega, el grupo deberá realizar una prueba de reproducibilidad:

1. partir de una copia limpia del proyecto;
2. conservar solamente los datos originales y el código;
3. ejecutar el procesamiento desde el comienzo;
4. regenerar los datos procesados;
5. regenerar las tablas;
6. regenerar las figuras;
7. generar nuevamente el informe Quarto;
8. verificar que no sea necesario modificar manualmente caminos o resultados intermedios.

Idea clave

Si el proyecto solamente funciona después de realizar manualmente una serie de pasos que no están documentados, todavía no es completamente reproducible.

18.36. Preguntas para orientar el desarrollo

Antes de finalizar, cada grupo debería poder responder:

1. ¿puede otra persona entender qué hace el proyecto?;
2. ¿puede identificar rápidamente cuál es la base original?;
3. ¿puede reconstruir la base procesada?;
4. ¿puede saber qué script debe ejecutar primero?;
5. ¿los gráficos pueden regenerarse?;
6. ¿los resultados del informe provienen del código?;
7. ¿existen pasos manuales innecesarios?;
8. ¿hay código repetido que debería transformarse en una función?;
9. ¿qué ocurre si aparece un dato inesperado?;
10. ¿las principales decisiones están documentadas?;
11. ¿las conclusiones están realmente apoyadas por los resultados?;

18.37. Cierre de la sección

En esta sección integramos los diferentes componentes estudiados durante el curso dentro de un proyecto reproducible.

Vimos que la programación para ciencia de datos no termina cuando conseguimos ejecutar un análisis.

También necesitamos organizar datos, código y resultados; automatizar procedimientos; documentar decisiones; verificar supuestos; controlar dependencias y comunicar los resultados de forma que el proceso pueda reconstruirse.

Quarto permite cerrar este flujo integrando texto, código, tablas y visualizaciones dentro de un documento generado automáticamente.

La actividad final propone justamente poner en práctica esta idea: transformar una colección de análisis en un proyecto organizado, automatizado, documentado y reproducible.